Microservices Architecture — Complete In-Depth Notes & Interview Preparation

---

Table of Contents

1. Introduction to Microservices
2. Monolithic vs Microservices Architecture
3. Advantages of Microservices
4. Disadvantages of Microservices
5. Key Principles of Microservices
6. Domain-Driven Design (DDD)
7. Database Per Service Pattern
8. Inter-Service Communication
9. API Gateway
10. Service Discovery (Eureka)
11. Config Server
12. Circuit Breaker Pattern
13. Load Balancing
14. Saga Pattern
15. CQRS Pattern
16. Event Sourcing
17. Distributed Tracing
18. Centralized Logging
19. 12-Factor App Methodology
20. Security in Microservices
21. Docker & Containerization
22. Kubernetes & Orchestration
23. Spring Cloud Ecosystem
24. Additional Design Patterns
25. Interview Questions & In-Depth Answers

---

1. Introduction to Microservices

What are Microservices?

Microservices is an **architectural style** where a large application is structured as a collection of **small, autonomous services**, each running in its own process, communicating with lightweight mechanisms (typically HTTP REST or messaging), and built around a **specific business capability**.

The term was popularized by **Martin Fowler** and **James Lewis** in 2014. It evolved from Service-Oriented Architecture (SOA) but with key differences — microservices are smaller, independently deployable, and avoid heavy middleware like ESBs (Enterprise Service Buses).

Core Characteristics

1. Componentization via Services In traditional applications, components are libraries that are linked into a single program. In microservices, components are services that communicate over the network. This means each component can be deployed independently.

2. Organized Around Business Capabilities Instead of dividing teams by technology layers (UI team, backend team, database team), microservices organize teams around business capabilities. A single team owns everything related to a business function — from UI to database.

```
Traditional Layers:              Microservices (Business Teams):
┌──────────────────┐             ┌─────────┐  ┌─────────┐  ┌─────────┐
│   UI Layer       │             │Product  │  │ Order   │  │Billing  │
├──────────────────┤             │ Team    │  │ Team    │  │ Team    │
│  Business Logic  │      →      │ UI      │  │ UI      │  │ UI      │
├──────────────────┤             │ Logic   │  │ Logic   │  │ Logic   │
│  Data Access     │             │ DB      │  │ DB      │  │ DB      │
└──────────────────┘             └─────────┘  └─────────┘  └─────────┘
```

3. Products, Not Projects In a project mindset, a team builds a feature and hands it off. In microservices, a team owns a service for its entire lifecycle — development, deployment, maintenance, and support. As Amazon puts it: **"You build it, you run it."**

4. Smart Endpoints and Dumb Pipes Business logic lives in the services (smart endpoints), not in the communication layer. The communication channel (HTTP, messaging) is kept simple (dumb pipes). This is the opposite of traditional SOA, where the ESB (Enterprise Service Bus) contained complex routing and transformation logic.

5. Decentralized Governance Each team can choose the best technology for their service. There is no mandate to use a single tech stack. One service can use Java, another Python, and another Go — as long as they communicate via agreed-upon APIs.

6. Decentralized Data Management Each service manages its own database. This is the "Database Per Service" pattern. No service directly accesses another service's database.

7. Infrastructure Automation Microservices rely heavily on CI/CD pipelines, automated testing, infrastructure as code (Docker, Kubernetes), and monitoring. Without automation, managing dozens of services would be impractical.

8. Design for Failure Since services communicate over the network, failures are expected. Services must be designed to handle failures gracefully using patterns like Circuit Breaker, Retry, Timeout, and Fallback.

Real-World Companies Using Microservices

Company	Journey
Netflix	Migrated from monolith after a major database corruption in 2008. Took ~2 years. Created many open-source tools (Eureka, Zuul, Hystrix).
Amazon	Transitioned to microservices in early 2000s. CEO Jeff Bezos mandated that all teams expose functionality through APIs. Led to AWS.
Uber	Started as a monolith, migrated to 2000+ microservices as they expanded to multiple cities and services.
Spotify	Uses "Squads" (small teams), each owning microservices. This enables rapid feature development.

---

2. Monolithic vs Microservices Architecture

Monolithic Architecture — Deep Dive

A monolithic application is built as a **single, indivisible unit**. All modules (user management, product catalog, order processing, billing) are part of one codebase, compiled into one artifact (e.g., a single WAR/JAR file), and deployed as one unit.

```
┌──────────────────────────────────────────────┐
│            MONOLITHIC APPLICATION             │
│                                               │
│  ┌────────────────┐  ┌────────────────┐       │
│  │ User Module    │  │ Product Module│       │
│  │  - Register    │  │  - Add Product│       │
│  │  - Login       │  │  - Search     │       │
│  │  - Profile     │  │  - Update     │       │
│  └────────────────┘  └────────────────┘       │
│                                               │
│  ┌────────────────┐  ┌────────────────┐       │
│  │ Order Module   │  │ Billing Module│       │
│  │  - Place Order│  │  - Invoice    │       │
│  │  - Track       │  │  - Payment    │       │
│  │  - Cancel      │  │  - Refund     │       │
│  └────────────────┘  └────────────────┘       │
│                                               │
│  ┌──────────────────────────────────────┐     │
│  │          SHARED DATABASE              │     │
│  │  users | products | orders | billing |     │
│  └──────────────────────────────────────┘     │
└──────────────────────────────────────────────┘
       Deployed as ONE single unit (one JAR/WAR)
```

How it works internally: When a user places an order, the Order Module calls the Product Module and Billing Module through **in-process function calls** (direct method invocations). Everything runs in the same process, sharing the same memory and database.

```java
// In a monolith — direct function call, fast, simple
public class OrderService {
    @Autowired
    private ProductService productService;  // Same process

    @Autowired
    private BillingService billingService;  // Same process

    public Order placeOrder(OrderRequest request) {
        Product product = productService.getProduct(request.getProductId());  // Function call
        billingService.createInvoice(order);  // Function call
        return order;
    }
}
```

Advantages of Monolith:

- Simple to develop, test, and deploy initially
- Single codebase — easy to navigate
- In-process calls are fast (no network overhead)
- Single database — easy transactions with ACID
- Good for small teams and simple applications

Problems of Monolith as it grows:

1. Deployment Fear As the codebase grows to millions of lines, even a small change requires deploying the entire application. A bug in the Billing Module means redeploying everything, risking breaking the Product Module.

2. Scaling Limitations If only the Product Search feature is getting heavy traffic, you cannot scale just that feature. You must scale the entire application, wasting resources.

```
Monolith Scaling:
Product Search needs more power → Scale entire app → Wasteful

┌──────────┐  ┌──────────┐  ┌──────────┐
│ Full App │  │ Full App │  │ Full App │
│(instance1│  │(instance2│  │(instance3│
│ ALL      │  │ ALL      │  │ ALL      │
│ modules) │  │ modules) │  │ modules) │
└──────────┘  └──────────┘  └──────────┘
  All 3 instances have ALL modules, even if only Product Search needs scaling
```

3. Technology Lock-in The entire application must use the same language, framework, and often the same database. If you want to use Python's ML libraries for recommendations, you can't — everything must be in Java (or whatever the monolith uses).

4. Team Bottlenecks 50 developers working on one codebase leads to merge conflicts, slow builds, long release cycles, and coordination overhead.

5. Single Point of Failure A memory leak in the Billing Module crashes the entire application. Users can't even browse products because everything runs in the same process.

Microservices Architecture — Deep Dive

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│   Product    │  │    Order     │  │   Customer   │  │   Billing    │
│   Service    │  │   Service    │  │   Service    │  │   Service    │
│              │  │              │  │              │  │              │
│ Spring Boot  │  │ Spring Boot  │  │ Spring Boot  │  │ Spring Boot  │
│ Port: 8081   │  │ Port: 8082   │  │ Port: 8083   │  │ Port: 8084   │
│              │  │              │  │              │  │              │
│  ┌────────┐  │  │  ┌────────┐  │  │  ┌────────┐  │  │  ┌────────┐  │
│  │ MySQL  │  │  │  │ MySQL  │  │  │  │MongoDB │  │  │  │ MySQL  │  │
│  │product │  │  │  │ order  │  │  │  │customer│  │  │  │billing │  │
│  │  _db   │  │  │  │  _db   │  │  │  │  _db   │  │  │  │  _db   │  │
│  └────────┘  │  │  └────────┘  │  │  └────────┘  │  │  └────────┘  │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘
      │                  │                  │                  │
      └──────── Communicate via REST APIs / Message Queues ──────┘
```

How it works internally: Each service is a **separate Spring Boot application** running in its own process. When Order Service needs product information, it makes an **HTTP REST call** over the network to Product Service.

```java
// In microservices — network call via OpenFeign
@FeignClient(name = "product-service")
public interface ProductServiceClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProduct(@PathVariable Long id);  // HTTP call over network
}

public class OrderService {
    @Autowired
    private ProductServiceClient productClient;  // Remote service

    public Order placeOrder(OrderRequest request) {
        ProductDTO product = productClient.getProduct(request.getProductId());  // Network call
```

```
        // ... create order
        return order;
    }
}
```

Key Difference: In a monolith, `productService.getProduct()` is a direct method call (nanoseconds). In microservices, `productClient.getProduct()` is an HTTP call over the network (milliseconds), which can fail, timeout, or return errors.

Detailed Comparison

Aspect	Monolith	Microservices
Architecture	Single unit, all modules together	Independent services, each separate
Codebase	One large repository	One repo per service (or mono-repo)
Database	Single shared database	Each service has its own database
Deployment	Entire app deployed at once	Each service deployed independently
Scaling	Scale the whole app	Scale individual services
Technology	Single tech stack	Different tech per service (polyglot)
Communication	In-process function calls	Network calls (REST, gRPC, MQ)
Transactions	ACID transactions easy	Distributed transactions (Saga Pattern)
Testing	Easier (everything local)	Harder (need other services running)
Debugging	Easy (single process, single log)	Hard (multiple processes, distributed logs)
Team Size	One large team	Small independent teams (2-pizza rule)
Failure	One module fails = entire app fails	One service fails = others continue
Initial Complexity	Low (simple to start)	High (infrastructure overhead)
Long-term Complexity	High (grows into "big ball of mud")	Manageable (each service stays small)

When to Choose What?

Start with Monolith when:

- You're building an MVP or prototype
- Team is small (< 8 developers)
- Domain is not well understood yet (boundaries unclear)
- Speed to market is more important than scalability
- Simple application with straightforward requirements

Move to Microservices when:

- Application has grown large and complex
- Different modules need different scaling strategies
- Multiple teams need to work independently
- You need to deploy different parts at different frequencies
- Different parts need different technology stacks
- You have the infrastructure maturity (CI/CD, monitoring, containerization)

Important: Many successful companies (Shopify, Stack Overflow) run on monoliths. Microservices add complexity — don't use them unless the benefits outweigh the costs. As Martin Fowler says: **"Don't start with microservices. Start with a monolith and extract microservices as the need arises."**

---

3. Advantages of Microservices

1. Independent Deployment

Each service can be built, tested, and deployed independently without affecting other services. This is perhaps the most significant advantage.

Why this matters: In a monolith, deploying a small bug fix in the billing module requires deploying the entire application. This means:

- Full regression testing of all modules
- Risk of breaking unrelated features
- Coordination with all teams
- Longer deployment windows

In microservices:

- Fix the bug in Billing Service
- Test only Billing Service
- Deploy only Billing Service
- Other services are untouched

Real-world impact: Amazon deploys to production every **11.7 seconds**. This is possible because each team deploys their own service independently.

2. Granular Scalability

You can scale individual services based on their specific resource needs, rather than scaling the entire application.

```
Example: E-commerce during a flash sale

Product Search Service → 10 instances (high read traffic)
Cart Service           → 5 instances (moderate traffic)
User Profile Service   → 1 instance (low traffic)
Payment Service        → 3 instances (moderate, but needs reliability)

Total resources used: What's actually needed

vs. Monolith:
10 instances of the ENTIRE app (even though only search needs 10)
Total resources used: 10x EVERYTHING → Wasteful and expensive
```

3. Fault Isolation (Resilience)

When one service fails, other services continue to operate. This is not possible in a monolith where a crash in one module brings down everything.

Example:

```
Billing Service encounters a database connection pool exhaustion.
- In Monolith: The entire application becomes unresponsive. Users can't even browse products.
- In Microservices: Billing Service is down, but:
  - Users can still browse products (Product Service is running)
  - Users can still add items to cart (Cart Service is running)
  - Users can still view their profile (User Service is running)
  - Only invoice generation is temporarily unavailable
  - Circuit Breaker returns: "Invoice will be generated shortly"
```

4. Technology Freedom (Polyglot)

Each service can use the programming language, framework, and database that best suits its requirements.

Example:

```
Product Service      → Java + Spring Boot + MySQL
                       (Strong typing, enterprise-grade CRUD)

Search Service       → Python + Elasticsearch
                       (Python's NLP libraries for smart search)

Real-time Chat       → Node.js + WebSocket + Redis
                       (Event-loop model perfect for real-time)

ML Recommendation    → Python + TensorFlow + MongoDB
                       (ML ecosystem is strongest in Python)

Report Generation    → Go + PostgreSQL
                       (Go's concurrency for parallel report generation)
```

5. Team Autonomy & Organizational Scalability

Small, cross-functional teams (5-8 people) own individual services end-to-end. This follows **Conway's Law**: "Organizations design systems that mirror their communication structure."

Amazon's Two-Pizza Rule: A team should be small enough to be fed by two pizzas (~6-8 people). Each team owns one or more microservices and has full autonomy over technology choices, deployment schedules, and development practices.

6. Faster Time to Market

Multiple teams can develop, test, and deploy features in parallel without waiting for other teams.

```
Monolith: Sequential releases
Week 1: Team A builds Product feature
Week 2: Team B builds Order feature (waits for Team A's merge)
Week 3: Integration testing of everything
Week 4: Deploy everything together
Total: 4 weeks for 2 features

Microservices: Parallel releases
Week 1-2: Team A builds & deploys Product feature
Week 1-2: Team B builds & deploys Order feature (simultaneously)
Total: 2 weeks for 2 features
```

7. Easier to Understand and Maintain

Each service is small and focused. A new developer joining the team only needs to understand one service (maybe 5,000-10,000 lines of code), not the entire application (which could be millions of lines).

8. Reusability

Services can be reused across different applications and contexts. A Notification Service built for one project can be used by any other project that needs email/SMS notifications.

---

4. Disadvantages of Microservices

1. Distributed System Complexity

This is the biggest challenge. You're now dealing with a **distributed system**, which introduces problems that don't exist in a monolith.

Network is unreliable: The network between services can fail, be slow, or drop packets. In a monolith, a function call always reaches its target. In microservices, an HTTP call might not.

The 8 Fallacies of Distributed Computing (Peter Deutsch):

1. The network is reliable → **It's NOT**
2. Latency is zero → **It's NOT**
3. Bandwidth is infinite → **It's NOT**
4. The network is secure → **It's NOT**
5. Topology doesn't change → **It DOES**
6. There is one administrator → **There ISN'T**
7. Transport cost is zero → **It's NOT**
8. The network is homogeneous → **It's NOT**

Every one of these assumptions will be violated in a microservices system, and you must design for it.

2. Data Consistency Challenges

In a monolith, you have ACID transactions:

```
BEGIN TRANSACTION;
  INSERT INTO orders (...);
  UPDATE inventory SET stock = stock - 20;
  INSERT INTO invoices (...);
COMMIT;
-- Either ALL succeed or ALL fail. Simple!
```

In microservices, each service has its own database. You **cannot** wrap a transaction across multiple databases. This means:

- Order is created in Order DB ✅
- Stock update in Inventory DB fails ❌
- Invoice in Billing DB never created ❌
- **Data is now inconsistent!**

Solution: We use **eventual consistency** instead of strong consistency. The Saga Pattern ensures that eventually all services reach a consistent state (with compensating actions for failures). But this adds significant complexity.

3. Testing Complexity

Unit testing is the same. But **integration testing** becomes much harder:

- You need all dependent services running
- You need their databases seeded with test data
- Network issues can cause flaky tests
- Test environments are expensive to maintain

Approaches to handle this:

- **Contract Testing** (Pact) — Verify that services agree on API contracts without running the actual service
- **Consumer-Driven Contract Testing** — Consumer defines the expected contract, provider verifies it
- **Test Containers** — Use Docker containers to spin up dependent services during testing
- **Service Virtualization** — Mock external services using tools like WireMock

4. Debugging and Troubleshooting

A single user request can flow through 5-10 services. When something goes wrong, finding the root cause is like finding a needle in a haystack.

```
User reports: "My order failed"

The request went through:
API Gateway → Auth Service → Order Service → Product Service → Inventory Service → Payment Service

Where did it fail? Why?
- Was it a timeout in Product Service?
- Was it a null response from Inventory Service?
- Was it a payment gateway error in Payment Service?
- Was it a network issue between services?

Without proper tooling (distributed tracing, centralized logging), debugging this is a nightmare.
```

5. Operational Overhead

Each service needs:

- Its own CI/CD pipeline
- Its own database
- Its own monitoring and alerting
- Its own log management
- Its own health checks
- Its own documentation

With 20 microservices, that's 20 of everything. You need a mature **DevOps culture** and automation to manage this.

6. Network Latency

In a monolith, calling another module is a function call (nanoseconds). In microservices, it's a network call (milliseconds).

```
Monolith:
getProduct() → 0.001ms (direct function call)

Microservices:
HTTP GET /products/123 → 5-50ms (network call)
  - DNS resolution
  - TCP handshake
  - Data serialization (object → JSON)
  - Network transfer
  - Data deserialization (JSON → object)
  - Processing
  - Response back through all layers

If a request goes through 5 services, that's 25-250ms just in network overhead.
```

7. Service Coordination

Features that span multiple services require careful coordination. For example, adding a new field to an order might require changes in Order Service, Product Service, and the API Gateway. This requires backward-compatible API changes, versioning, and coordination between teams.

8. Infrastructure Cost

More services = more servers, more databases, more load balancers, more monitoring tools. The infrastructure cost is significantly higher than a monolith.

---

5. Key Principles of Microservices

1. Single Responsibility Principle (SRP)

Each service should have **one, and only one, reason to change**. It should encapsulate a single business capability.

```
✅ Good:
Product Service → Manages product catalog (CRUD, search, categories)
Order Service   → Manages order lifecycle (create, track, cancel, return)
Payment Service → Manages payments (charge, refund, reconciliation)

❌ Bad:
ProductAndOrderService → Manages products AND orders
(Two reasons to change: product logic changes OR order logic changes)
```

How to identify if a service is doing too much:

- If changes in one area frequently require changes in another area within the same service
- If the service has multiple databases or database schemas
- If different parts of the service have different scaling needs
- If different teams need to work on different parts of the same service

2. Loose Coupling

Services should be **independent** of each other. Changing the internal implementation of one service should not require changes in other services.

Tight Coupling (Bad):

```
Order Service directly reads Product Service's database:
- Product Service changes table structure → Order Service breaks
- Product Service migrates to MongoDB → Order Service breaks
- Product Service is down → Order Service can't even read static data
```

Loose Coupling (Good):

```
Order Service calls Product Service's API:
- Product Service changes internal structure → API stays same → Order Service fine
- Product Service migrates DB → API stays same → Order Service fine
- Product Service is down → Circuit Breaker → fallback response
```

How to achieve loose coupling:

- Communicate only through well-defined APIs
- Never share databases
- Use asynchronous messaging where possible
- Use events instead of direct commands ("OrderPlaced" event vs "ReduceStock" command)

3. High Cohesion

Related functionality should be grouped together within a service. Everything about a business concept should live in one place.

```
✅ High Cohesion:
Product Service contains:
  - Product CRUD
  - Product categories
  - Product pricing
  - Product search
  (All product-related → high cohesion)

❌ Low Cohesion:
Product Service contains:
  - Product CRUD
  - User authentication    ← Not related to products
  - Email notifications    ← Not related to products
```

4. Design for Failure

In a distributed system, things **will** fail. Design every service to handle failures gracefully.

Patterns for failure handling:

- **Timeout** — Don't wait forever for a response (set max wait time)
- **Retry** — Try again for transient failures (with exponential backoff)
- **Circuit Breaker** — Stop calling a failing service after threshold
- **Fallback** — Return a default/cached response when service is unavailable
- **Bulkhead** — Isolate resources so one failure doesn't exhaust everything

5. Decentralized Everything

- **Decentralized Data** — Each service owns its data
- **Decentralized Governance** — Teams choose their own tools
- **Decentralized Development** — Teams work independently

- **Decentralized Deployment** — Services deploy independently

6. Evolutionary Design

Services should be designed to be **replaceable**. If a service becomes too complex, you should be able to rewrite it from scratch without affecting the rest of the system. This is possible because the service boundary (its API) acts as a contract that other services depend on, not the internal implementation.

---

6. Domain-Driven Design (DDD)

Why DDD Matters for Microservices

The hardest part of microservices is **defining service boundaries**. How do you decide what should be a separate service? DDD provides a structured approach to this problem.

Key DDD Concepts

1. Domain The subject area your software is built for. For our project, the domain is "Car Spare Parts Business."

2. Subdomain A specific area within the domain. Our domain has subdomains:

- Product Management
- Inventory Management
- Customer Management
- Order Processing
- Billing & Payments

3. Bounded Context A boundary within which a particular model is defined and applicable. Each bounded context typically maps to one microservice.

```
┌──────────────────────────────────────────────────────────────┐
│                    CAR SPARE PARTS DOMAIN                    │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │   Product    │  │  Inventory   │  │  Customer    │       │
│  │  Bounded     │  │  Bounded     │  │  Bounded     │       │
│  │  Context     │  │  Context     │  │  Context     │       │
│  │              │  │              │  │              │       │
│  │ - Product    │  │ - Stock      │  │ - Customer   │       │
│  │ - Category   │  │ - Warehouse  │  │ - Address    │       │
│  │ - Brand      │  │ - Supplier   │  │ - Credit     │       │
│  │ - Vehicle    │  │ - Reorder    │  │ - Type       │       │
│  │   Compat.    │  │   Level      │  │   (retail/   │       │
│  │              │  │              │  │   wholesale)│       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐                         │
│  │   Order      │  │  Billing     │                         │
│  │  Bounded     │  │  Bounded     │                         │
│  │  Context     │  │  Context     │                         │
│  │              │  │              │                         │
│  │ - Order      │  │ - Invoice    │                         │
│  │ - OrderItem  │  │ - Payment    │                         │
│  │ - Status     │  │ - Credit     │                         │
│  │ - Return     │  │   Track      │                         │
│  └──────────────┘  └──────────────┘                         │
└──────────────────────────────────────────────────────────────┘
```

4. Ubiquitous Language Within each bounded context, everyone (developers, business people) uses the same language. The word "Product" in the Product Context means a spare part with its specifications. The word "Product" in the Order Context might just mean a line item with an ID and quantity. Same word, different meaning in different contexts — and that's okay.

5. Aggregates A cluster of domain objects treated as a single unit. For example, an Order aggregate contains Order, OrderItems, and OrderStatus. All changes to order items go through the Order aggregate root.

6. Context Mapping How bounded contexts relate to each other:

```
Product Context ←── uses data from ──→ Order Context
                    (Product ID, name, price)

Customer Context ←── uses data from ──→ Order Context
                     (Customer ID, name)

Order Context ←── triggers ──→ Billing Context
                  (Order completed → Generate invoice)

Order Context ←── triggers ──→ Inventory Context
                  (Order placed → Reduce stock)
```

How to Define Service Boundaries

Rule of thumb:

- One Bounded Context = One Microservice
- Each service should be independently deployable
- Each service should have its own database
- Changes to one service should rarely require changes to another
- A service should be small enough for one team to own

Warning signs of wrong boundaries:

- Two services that always change together → Maybe they should be one service
- One service that changes for multiple unrelated reasons → Maybe it should be split
- Excessive inter-service communication → Boundaries might be wrong

---

7. Database Per Service Pattern

The Pattern

Each microservice has its **own dedicated database** that is not accessible by any other service. If a service needs data from another service, it must go through that service's API.

Why This Is Non-Negotiable

This is not just a "nice to have" — it's fundamental to microservices. Without it, services are not truly independent.

Scenario: Shared Database (Anti-Pattern)

```
Product Service ──→ ┌──────────┐ ←── Order Service
                    │ SHARED   │
Customer Service ──→│ DATABASE │ ←── Billing Service
```

```
                    └──────────┘

Problems:
1. Product Service wants to change the "products" table schema
   → Must coordinate with Order Service (which also reads the table)
   → Must coordinate with Billing Service (which joins with products for invoices)
   → Simple schema change becomes a cross-team effort

2. Product Service wants to migrate from MySQL to MongoDB
   → Cannot! Other services depend on the MySQL schema
   → Technology lock-in at the database level

3. Product Service database is under heavy load
   → All services suffer because they share the same database
   → Cannot scale Product DB independently

4. A bug in Customer Service runs a bad query that locks the products table
   → Product Service is affected by a bug in a completely different service
```

Scenario: Database Per Service (Correct)

```
Product Service → [product_db]     (MySQL)
Order Service   → [order_db]       (MySQL)
Customer Service→ [customer_db]    (MongoDB)
Billing Service → [billing_db]     (PostgreSQL)

Benefits:
1. Product Service changes its schema → No impact on others
2. Customer Service uses MongoDB → No problem
3. Product DB under load → Scale just that DB
4. Bug in Customer DB → Only Customer Service affected
```

How Services Share Data

Pattern 1: API Composition When a service needs data from another service, it makes an API call.

```
Order Service needs product name and price:

Order Service → GET /api/products/123 → Product Service
              ← { "name": "Oil Filter", "price": 250 }

Order Service stores only: productId, productName, price (at time of order)
in its OWN database.
```

Pattern 2: Data Duplication (Denormalization) Services store a copy of the data they frequently need. The copy is updated via events.

```
Product Service: { id: 123, name: "Oil Filter", price: 250, brand: "Bosch", ... }
                      │
                      │ publishes "ProductPriceUpdated" event
                      ▼
Order Service stores: { productId: 123, productName: "Oil Filter", price: 250 }
                      (Only the fields it needs, updated via events)
```

Pattern 3: CQRS for Cross-Service Queries For complex queries spanning multiple services, build a separate read-optimized view.

```
Need a report: "All orders with product details and customer info"

Instead of querying 3 databases:
→ Build a Report Service that subscribes to events from all services
→ Builds a denormalized view optimized for reporting
→ Query this view for reports
```

Choosing the Right Database

Service	Best Database	Why
Product Service	MySQL/PostgreSQL	Structured data, relationships (product-category-brand)
Search Service	Elasticsearch	Full-text search, fuzzy matching
Customer Service	MongoDB	Flexible schema (different customer types have different fields)
Session/Cache	Redis	Fast key-value lookups, TTL support
Order Service	MySQL/PostgreSQL	ACID transactions for order integrity
Analytics	ClickHouse/BigQuery	Columnar storage for fast aggregation queries

Challenges and Solutions

Challenge	Solution
Cross-service transactions	Saga Pattern
Cross-service queries	API Composition or CQRS
Data duplication	Event-driven sync (accept some duplication)
Referential integrity	Application-level checks (not DB foreign keys)
Reporting across services	Dedicated Reporting/Analytics Service

---

8. Inter-Service Communication

Overview

In a monolith, modules communicate via direct function calls. In microservices, services are separate processes (possibly on different machines), so they communicate over the network. There are two fundamental approaches.

1. Synchronous Communication

The caller sends a request and **blocks (waits)** until it gets a response.

REST (Representational State Transfer)

The most common approach. Uses HTTP protocol with standard methods.

```
HTTP Methods:
GET    → Read data       → GET /api/products/123
POST   → Create data     → POST /api/products (body: {...})
PUT    → Update data     → PUT /api/products/123 (body: {...})
DELETE → Delete data     → DELETE /api/products/123
```

Example Flow:

```
Order Service needs product info:

Order Service                          Product Service
     |                                       |
     |── GET /api/products/123 ──────────────→|
     |                                       |── Query product_db
     |                                       |← Result
     |←── 200 OK {"name":"Oil Filter"} ──────|
     |                                       |
     (Order Service was BLOCKED during this time)
```

Spring Boot Tools for REST calls:

a) RestTemplate (Legacy, not recommended for new projects)

```java
RestTemplate restTemplate = new RestTemplate();
Product product = restTemplate.getForObject(
    "http://product-service/api/products/123", Product.class);
```

b) WebClient (Reactive, non-blocking)

```java
WebClient webClient = WebClient.create("http://product-service");
Mono<Product> product = webClient.get()
    .uri("/api/products/123")
    .retrieve()
    .bodyToMono(Product.class);
```

c) OpenFeign (Declarative, recommended)

```java
@FeignClient(name = "product-service")
public interface ProductServiceClient {

    @GetMapping("/api/products/{id}")
    Product getProduct(@PathVariable Long id);

    @GetMapping("/api/products")
    List<Product> getAllProducts();
}

// Usage — looks like a regular method call!
Product product = productClient.getProduct(123L);
```

OpenFeign is preferred because it's **declarative** — you define an interface, and Spring generates the implementation. It also integrates with Eureka for service discovery and Spring Cloud LoadBalancer for load balancing.

gRPC (Google Remote Procedure Call)

A high-performance RPC framework that uses **Protocol Buffers** for serialization (binary format, much faster than JSON).

```
REST:  JSON text → {"name": "Oil Filter", "price": 250} → ~50 bytes
gRPC:  Binary    → compressed binary data                → ~15 bytes
```

When to use gRPC over REST:

- Internal service-to-service communication (not client-facing)
- High-throughput, low-latency requirements
- Streaming data (gRPC supports bidirectional streaming)

2. Asynchronous Communication

The caller sends a message and **does not wait** for a response. The message goes into a **message broker** (queue), and the consumer processes it when ready.

Message Queue Pattern

```
Producer → [Message Broker] → Consumer

Order Service publishes: "OrderPlaced" event
                                │
                     ┌──────────┼──────────┐
                     ↓          ↓          ↓
               Inventory   Billing   Notification
               Service     Service   Service
               (reduce     (create   (send email
                stock)     invoice)  to customer)
```

RabbitMQ vs Apache Kafka

Feature	RabbitMQ	Apache Kafka
Type	Message Broker	Event Streaming Platform
Model	Push (broker pushes to consumers)	Pull (consumers pull from broker)
Message Retention	Deleted after consumed	Retained for configured duration (days/weeks)
Ordering	Per queue	Per partition (guaranteed within partition)
Throughput	Thousands/sec	Millions/sec
Use Case	Task queues, RPC, routing	Event streaming, log aggregation, real-time analytics
Replay	Not possible (message deleted)	Possible (messages retained)
Complexity	Simpler to set up	More complex (Zookeeper, partitions)
Best For	Traditional messaging, moderate load	High-throughput event streaming

When to use RabbitMQ:

- Task distribution (send email, generate PDF)
- Request-reply pattern (RPC)

- Complex routing (topic exchange, headers exchange)
- Moderate message volume

When to use Kafka:

- Event streaming (real-time data pipelines)
- Event sourcing (storing events as the source of truth)
- Log aggregation
- High-volume message processing
- When you need to replay events

Event-Driven Architecture

Instead of services calling each other directly, services **publish events** when something happens, and interested services **subscribe** to those events.

```
Order Service: (something happens) → publishes "OrderPlaced" event
                                          │
              ┌──────────────────────────┼──────────────────────┐
              ↓                          ↓                      ↓
     Inventory Service          Billing Service        Notification
     (Subscribed to             (Subscribed to         Service
      "OrderPlaced")            "OrderPlaced")         (Subscribed to
     → Reduces stock            → Creates invoice       "OrderPlaced")
     → Publishes                → Publishes             → Sends email
       "StockReduced"            "InvoiceCreated"
```

Benefits of Event-Driven:

- Services are truly decoupled (publisher doesn't know who subscribes)
- Easy to add new consumers without changing the publisher
- Naturally asynchronous and scalable
- If a consumer is down, events wait in the queue

Choosing Communication Style

Scenario	Style	Why
Check product availability before order	**Sync (REST)**	Need answer NOW before proceeding
Verify customer credit before order	**Sync (REST)**	Must validate before accepting order
Update stock after order	**Async (MQ)**	Can happen in background
Generate invoice after order	**Async (MQ)**	Can happen in background
Send notification	**Async (MQ)**	Non-critical, can be delayed
Real-time price check	**Sync (REST)**	User is waiting for the price
Analytics/Reporting	**Async (MQ)**	Not time-sensitive

Best Practice: Use Both

```
Order Placement Flow:
```

```
1. [SYNC] Order Service → Product Service: "Is product available?"
   ← "Yes, price is 250"

2. [SYNC] Order Service → Customer Service: "Is customer valid?"
   ← "Yes, wholesale customer with credit"

3. Order Service creates order in its DB

4. [ASYNC] Order Service publishes "OrderPlaced" event
   → Inventory Service: reduces stock
   → Billing Service: generates invoice
   → Notification Service: sends confirmation email
```

---

9. API Gateway

What is an API Gateway?

An API Gateway is a server that acts as a **single entry point** for all client requests into a microservices system. It sits between external clients and the internal microservices, handling cross-cutting concerns like routing, authentication, rate limiting, and more.

Think of it as the **receptionist** at a large office building — visitors don't wander around looking for the right department. They go to the reception, state their purpose, and the receptionist directs them to the right place.

The Problem It Solves

Without an API Gateway:

```
Mobile App
  │
  ├── GET http://10.0.1.5:8081/api/products/123    (Product Service)
  ├── GET http://10.0.1.6:8082/api/orders           (Order Service)
  ├── GET http://10.0.1.7:8083/api/customers/456    (Customer Service)
  └── POST http://10.0.1.8:8084/api/billing         (Billing Service)

Problems:
1. Client must know every service address
2. If a service moves to a different server, client must update
3. No centralized authentication — each service must validate tokens
4. No rate limiting — a buggy client can flood services
5. Client makes 4 separate HTTP calls → slow on mobile networks
6. Internal service structure is exposed to the outside world → security risk
7. CORS must be configured on every service
```

With an API Gateway:

```
Mobile App
  │
  └── ALL requests → http://api.example.com/...
                        │
                    ┌───┴──────────────────┐
                    │      API GATEWAY     │
                    │                      │
                    │ ✓ Single entry point │
                    │ ✓ Route to correct svc │
                    │ ✓ Authenticate once  │
                    │ ✓ Rate limit         │
                    │ ✓ Aggregate responses │
                    │ ✓ Hide internal struct │
```

```
                    └──────────┬──────────┘
                               │
                               │
            ┌──────────────────┼──────────────────┐
            ↓                  ↓                  ↓
      Product Svc      Order Svc      Billing Svc
```

Responsibilities in Detail

1. Request Routing Maps incoming request paths to the correct microservice.

```
/api/products/**  → Product Service (localhost:8081)
/api/orders/**    → Order Service (localhost:8082)
/api/customers/** → Customer Service (localhost:8083)
/api/billing/**   → Billing Service (localhost:8084)
```

2. Authentication & Authorization Validates JWT tokens at the gateway level. If the token is invalid, the request is rejected before it reaches any microservice.

```
Client sends: Authorization: Bearer <JWT_TOKEN>
Gateway: Validates token → Valid? Route to service : Return 401 Unauthorized
```

3. Rate Limiting Limits how many requests a client can make in a given time period to prevent abuse.

```
Rule: Max 100 requests per minute per client
Client sends request #101 in the same minute → Gateway returns 429 Too Many Requests
```

4. Load Balancing If a service has multiple instances, the gateway distributes requests across them.

```
3 instances of Product Service:
Request 1 → Instance 1
Request 2 → Instance 2
Request 3 → Instance 3
```

5. Response Aggregation (API Composition) A single client request might need data from multiple services. The gateway can make multiple internal calls and combine the responses.

```
Client: GET /api/order-details/123

Gateway internally calls:
  → Order Service: GET /orders/123
  → Product Service: GET /products/456
  → Customer Service: GET /customers/789

Gateway combines responses into one:
{
  "order": { ... },
  "product": { ... },
  "customer": { ... }
}
Client gets ONE response instead of making 3 calls.
```

6. Circuit Breaking If a downstream service is failing, the gateway can short-circuit requests and return a fallback response instead of waiting for a timeout.

7. Request/Response Transformation Transform request headers, add correlation IDs, convert between protocols (e.g., external HTTPS to internal HTTP).

8. Caching Cache frequent, rarely-changing responses (like product catalog) at the gateway level to reduce load on services.

9. SSL Termination Handle HTTPS at the gateway. Internal communication between gateway and services can be plain HTTP (within a trusted network), reducing encryption/decryption overhead on individual services.

10. Logging & Monitoring Log all incoming requests for auditing and monitoring. Since all traffic flows through the gateway, it's the perfect place to observe traffic patterns.

Architecture with Spring Cloud Gateway

```yaml
# application.yml for Spring Cloud Gateway
spring:
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://PRODUCT-SERVICE      # lb = load balanced, uses Eureka
          predicates:
            - Path=/api/products/**      # Match this URL pattern
          filters:
            - StripPrefix=1              # Remove /api prefix

        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/api/orders/**

        - id: customer-service
          uri: lb://CUSTOMER-SERVICE
          predicates:
            - Path=/api/customers/**
```

API Gateway vs Load Balancer

Feature	API Gateway	Load Balancer
Primary Role	Request routing + cross-cutting concerns	Distribute traffic across instances
Intelligence	Content-based routing, transformation	Simple distribution algorithms
Authentication	Yes	No
Rate Limiting	Yes	Limited
Protocol	HTTP/HTTPS (Layer 7)	TCP/HTTP (Layer 4/7)
Aggregation	Can compose multiple service calls	No

Tools

Tool	Type	Best For
Spring Cloud Gateway	Java-native	Spring Boot microservices
Kong	Platform-agnostic	Plugin-based, enterprise features

Tool	Type	Best For
NGINX	Web server / reverse proxy	High-performance routing
AWS API Gateway	Managed service	AWS-native applications
Envoy	Service proxy	Kubernetes, service mesh (Istio)

---

10. Service Discovery (Eureka)

The Problem

In a microservices architecture, services need to find each other to communicate. But services are dynamic — they can:

- **Move** to different servers
- **Change** ports
- **Scale** up (new instances added) or down (instances removed)
- **Crash** and restart on a different host

Hardcoding service addresses (like `http://192.168.1.10:8081`) doesn't work because addresses change constantly.

What is Service Discovery?

Service Discovery is the automatic detection and registration of services in a network. It maintains a **registry** — a live database of all running service instances with their network locations.

How Netflix Eureka Works

Eureka follows the **client-side discovery** pattern:

```
┌──────────────────────────────────────────────────────────┐
│                    EUREKA SERVER                         │
│                                                          │
│   Service Registry:                                      │
│   ┌──────────────────────────────────────────┐          │
│   │ Service Name    | Instances              |          │
│   ├─────────────────┼────────────────────────┤          │
│   │ PRODUCT-SERVICE | 192.168.1.10:8081      |          │
│   │                 | 192.168.1.11:8081      |          │
│   │ ORDER-SERVICE   | 192.168.1.12:8082      |          │
│   │ BILLING-SERVICE | 192.168.1.13:8083      |          │
│   └──────────────────────────────────────────┘          │
└──────────────────────────────────────────────────────────┘
```

Step-by-step process:

Step 1: Service Registration When a service starts up, it sends a registration request to Eureka with:

- Service name (e.g., "PRODUCT-SERVICE")
- Host and port (e.g., "192.168.1.10:8081")
- Health check URL
- Metadata

Step 2: Heartbeat (Renewal) Every 30 seconds (default), each service sends a heartbeat to Eureka: "I'm still alive and healthy."

If Eureka doesn't receive a heartbeat for 90 seconds (3 missed heartbeats), it considers the service as dead and removes it from the registry.

Step 3: Service Discovery (Fetching Registry) When Order Service needs to call Product Service:

1. Order Service asks Eureka: "Give me all instances of PRODUCT-SERVICE"
2. Eureka responds: ["192.168.1.10:8081", "192.168.1.11:8081"]
3. Order Service caches this list locally (refreshed every 30 seconds)
4. Order Service uses a load balancer to pick one instance

Step 4: De-registration When a service shuts down gracefully, it sends a de-registration request to Eureka, which immediately removes it from the registry.

Self-Preservation Mode

This is an important concept that is often asked in interviews.

Problem: If there's a network partition (Eureka can't reach services, but services are actually running), Eureka might remove all services from the registry, thinking they're all dead. This would be catastrophic.

Solution: Self-Preservation Mode If Eureka notices that more than 15% of services have stopped sending heartbeats, it assumes the problem is **network-related**, not that services are actually down. In this mode:

- Eureka stops removing instances from the registry
- It preserves the existing registry (hence "self-preservation")
- It shows a warning: "EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP"

Why it matters: It's better to route to a potentially dead instance (which will fail fast) than to route to NO instance (which definitely fails). Self-preservation prevents mass de-registration during network issues.

Eureka Server Configuration

```yaml
# Eureka Server - application.yml
server:
  port: 8761

eureka:
  client:
    register-with-eureka: false    # Server doesn't register with itself
    fetch-registry: false          # Server doesn't need to fetch registry
  server:
    enable-self-preservation: true # Enable self-preservation mode
```

Eureka Client Configuration

```yaml
# Any Microservice - application.yml
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    hostname: localhost
    prefer-ip-address: true
```

Client-Side vs Server-Side Discovery

Aspect	Client-Side (Eureka)	Server-Side (AWS ALB, K8s)
Who discovers?	The calling service	A dedicated load balancer/router
Registry cached at	Client side	Server side
Load balancing done by	Client	Server/Router
Advantage	No extra network hop	Client is simpler, doesn't need discovery logic
Disadvantage	Client needs discovery library	Extra network hop through router
Example	Netflix Eureka + Spring Cloud LoadBalancer	Kubernetes Service, AWS ALB

---

11. Config Server

The Problem in Detail

Consider a production environment with 10 microservices, each deployed across 3 environments (dev, staging, prod):

```
10 services × 3 environments = 30 configuration files to manage

Each file contains:
- Database connection URL and credentials
- Service port
- Eureka server URL
- Logging level
- Feature flags
- Third-party API keys
- Timeout values
- Thread pool sizes
```

Without Config Server:

- Configurations are scattered across 30 files
- Changing a database password requires updating 10 files and redeploying 10 services
- No version history of configuration changes
- No audit trail of who changed what
- Risk of configuration drift (dev and prod configs diverge)
- Sensitive data (passwords, API keys) stored in plain text in code repos

What Config Server Solves

Centralized Configuration: All configuration in one place (a Git repository).

Version Control: Every change is tracked in Git with commit history.

Environment-Specific: Same service, different config per environment.

Runtime Refresh: Change configuration without redeploying the service.

Encryption: Sensitive values stored encrypted, decrypted at runtime.

Architecture

```
┌──────────────────┐
│    Git Repo      │ (GitHub, GitLab, Bitbucket)
│                  │
│  ├── application.yml           (shared config)
│  ├── product-service.yml       (product-specific)
│  ├── product-service-dev.yml   (product dev)
│  ├── product-service-prod.yml  (product prod)
│  ├── order-service.yml         (order-specific)
│  ├── order-service-dev.yml     (order dev)
│  └── order-service-prod.yml    (order prod)
│                  │
└──────────────────┘
         │
         │ (reads from Git)
         ▼
┌──────────────────┐
│  Config Server   │ (Spring Cloud Config Server)
│  Port: 8888      │
└──────────────────┘
         │
         │ (services fetch config via HTTP)
         │
    ┌────┼────┬────────┐
    ▼    ▼    ▼        ▼
  Prod  Order Billing  Inventory
  Svc   Svc   Svc      Svc
```

Configuration Priority

Spring Cloud Config follows a specific priority order for resolving configuration:

```
Highest Priority:
1. product-service-prod.yml    (service-specific + profile)
2. product-service.yml         (service-specific)
3. application-prod.yml        (shared + profile)
4. application.yml             (shared default)
Lowest Priority

If the same key exists in multiple files, the highest priority wins.
```

Runtime Refresh with Spring Cloud Bus

Normally, changing a configuration requires restarting the service. With Spring Cloud Bus + RabbitMQ/Kafka, you can refresh config at runtime:

```
1. Developer changes config in Git repo
2. Webhook triggers Config Server
3. Config Server publishes "RefreshEvent" to Spring Cloud Bus (RabbitMQ)
4. All services subscribed to the bus receive the event
5. Services refresh their configuration WITHOUT restart

Git Repo → Config Server → [RabbitMQ Bus] → All Services refresh
```

Boot Order

Config Server must start **before** all other services because they depend on it for configuration.

```
Start Order:
1. Config Server    → Starts first (has its own local config)
2. Eureka Server    → Fetches its config from Config Server
3. API Gateway      → Fetches config, registers with Eureka
4. Business Services→ Fetch config, register with Eureka
```

What Happens If Config Server Goes Down?

- Services that are already running continue to work (they cache their configuration locally)
- New services or restarting services cannot fetch configuration
- **Solution:** Run multiple instances of Config Server for high availability

---

12. Circuit Breaker Pattern

The Problem: Cascading Failure

This is one of the most dangerous failures in a microservices system. One service going down can bring down the entire system.

Detailed Scenario:

```
Normal operation:
Client → Gateway → Order Service → Product Service → Response in 50ms ✅

Product Service becomes slow (database overloaded):
Client → Gateway → Order Service → Product Service → ...waiting...10 seconds...timeout

What happens in Order Service while waiting:
- Thread 1: Waiting for Product Service response... (blocked)
- Thread 2: Waiting for Product Service response... (blocked)
- Thread 3: Waiting for Product Service response... (blocked)
...
- Thread 200: Waiting for Product Service response... (blocked)

ORDER SERVICE HAS RUN OUT OF THREADS!
Now Order Service cannot handle ANY requests — even those that don't need Product Service.

Gateway notices Order Service is not responding:
- Gateway threads start getting blocked waiting for Order Service
- Gateway runs out of threads
- ENTIRE SYSTEM IS DOWN because of one slow database in Product Service!

This is Cascading Failure:
Product DB slow → Product Service slow → Order Service exhausted → Gateway exhausted → System DOWN
```

How Circuit Breaker Solves This

The Circuit Breaker wraps calls to external services. It monitors failures and "opens the circuit" when failures exceed a threshold, preventing the cascade.

The Three States — Detailed:

```
┌──────────────────────────────────────────────────────────┐
│                    CIRCUIT BREAKER                       │
│                                                          │
│  ┌──────────┐      failure threshold      ┌──────────┐   │
```

```
│  │          │      exceeded             │          │      │
│  │  CLOSED  │──────────────────────────→│   OPEN   │      │
│  │ (normal) │                           │ (blocked)│      │
│  │          │                           │          │      │
│  └──────────┘                           └──────────┘      │
│       ↑                                      │            │
│       │ success                    wait timeout expires   │
│       │                                      │            │
│       │              ┌──────────┐            │            │
│       │              │          │            │            │
│       └──────────────│ HALF-OPEN │←──────────┘            │
│          success     │  (testing)│                        │
│                      │          │──────────┐              │
│                      └──────────┘  failure │              │
│                                      │     │              │
│                                      │     ▼              │
│                                      │  ┌──────────┐      │
│                                      └→ │   OPEN   │      │
│                                         └──────────┘      │
└──────────────────────────────────────────────────────────┘
```

CLOSED State (Normal):

- All requests pass through to the target service
- Circuit Breaker monitors and counts failures
- If failures exceed threshold (e.g., 5 failures in 10 seconds) → transitions to OPEN
- Success calls reset the failure count

OPEN State (Blocking):

- No requests are sent to the target service
- Immediately returns a **fallback response**
- After a configured wait duration (e.g., 30 seconds) → transitions to HALF-OPEN
- This prevents thread exhaustion and gives the failing service time to recover

HALF-OPEN State (Testing):

- Allows a limited number of test requests through (e.g., 3 requests)
- If these test requests succeed → transitions to CLOSED (service recovered!)
- If any test request fails → transitions back to OPEN (still failing)

Configuration Parameters

Parameter	Description	Example
Failure Rate Threshold	% of failures to trip the circuit	50% (if half the calls fail, open the circuit)
Minimum Number of Calls	Minimum calls before evaluating failure rate	10 (don't evaluate until at least 10 calls)
Wait Duration in Open	How long to wait before trying HALF-OPEN	30 seconds
Permitted Calls in Half-Open	How many test calls in HALF-OPEN	3
Sliding Window Size	Window for counting failures	10 calls or 60 seconds

Fallback Strategies

When the circuit is open, don't just throw an error. Provide a meaningful fallback:

```
Strategy 1: Return cached data
  → Last known product price from cache
  → "Showing prices as of 10 minutes ago"

Strategy 2: Return default data
  → Default list of popular products
  → "Unable to load personalized recommendations, showing popular items"

Strategy 3: Return error with helpful message
  → "Product details temporarily unavailable. Your order has been queued."

Strategy 4: Call alternative service
  → Primary recommendation engine down? Use simplified recommendation logic
```

Related Resilience Patterns

Circuit Breaker is often used together with these patterns:

1. Retry Automatically retry a failed call (for transient errors like momentary network glitch).

```
Call fails → Wait 100ms → Retry → Wait 200ms → Retry → Wait 400ms → Retry → Give up
(Exponential backoff: each wait is 2x the previous)
```

Important: Only retry on **transient** errors (timeout, network error). Don't retry on **permanent** errors (404 Not Found, 400 Bad Request).

2. Timeout Set a maximum time to wait for a response. Don't wait indefinitely.

```
Order Service → Product Service (timeout: 3 seconds)
If no response in 3 seconds → Timeout exception → Trigger fallback
```

3. Bulkhead Isolate resources (thread pools) for different service calls so one slow service doesn't exhaust all threads.

```
Order Service has:
  Thread Pool A (10 threads) → For Product Service calls
  Thread Pool B (10 threads) → For Billing Service calls
  Thread Pool C (10 threads) → For Customer Service calls

Product Service is slow → Pool A exhausted
But Pool B and C are fine → Billing and Customer calls still work!
```

4. Rate Limiter Limit the number of calls to a service per time period.

```
Max 100 calls/second to Product Service
Call #101 → Rejected → Return fallback
```

Tool: Resilience4j (Spring Boot)

Resilience4j is the recommended library for Spring Boot (Hystrix from Netflix is deprecated).

---

13. Load Balancing

What is Load Balancing?

Load balancing distributes incoming network traffic across multiple instances of a service to ensure no single instance becomes overwhelmed. It improves availability, reliability, and performance.

Why Is It Needed?

```
Without Load Balancing:
Product Service has 3 instances, but all traffic goes to Instance 1:

Instance 1: ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 100% load → CRASHES!
Instance 2: ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   0% load → Idle
Instance 3: ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   0% load → Idle

With Load Balancing:
Instance 1: ▓▓▓▓▓▓▓▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  33% load → Healthy
Instance 2: ▓▓▓▓▓▓▓▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  33% load → Healthy
Instance 3: ▓▓▓▓▓▓▓▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  33% load → Healthy
```

Server-Side Load Balancing

A dedicated load balancer sits between the client and service instances.

```
Client Request ──→ ┌──────────────────┐
                   │   Load Balancer  │
                   │  (Nginx / AWS ELB)│
                   └──────────┬───────┘
                              │
                    ┌─────────┴─────────┐
                    ↓         ↓         ↓
              Instance1  Instance2  Instance3
```

Pros:

- Client doesn't need any discovery logic
- Centralized control over routing rules
- SSL termination at the load balancer

Cons:

- Extra network hop (client → LB → service)
- Load balancer is a single point of failure (unless HA)
- Must be provisioned and maintained

Examples: Nginx, HAProxy, AWS ALB/ELB, F5

Client-Side Load Balancing

The calling service itself decides which instance to call. It fetches the list of available instances from the service registry (Eureka) and applies a load balancing strategy locally.

```
Order Service (has Spring Cloud LoadBalancer):
  1. Fetches instance list from Eureka:
```

```
   PRODUCT-SERVICE → [Instance1:8081, Instance2:8082, Instance3:8083]

2. Applies load balancing strategy (Round Robin):
   Request 1 → Instance1:8081
   Request 2 → Instance2:8082
   Request 3 → Instance3:8083
   Request 4 → Instance1:8081 (back to first)

3. Makes direct call to chosen instance (no middleman)
```

Pros:

- No extra network hop
- No single point of failure
- Instance list is cached locally

Cons:

- Every service needs the load balancing library
- Logic is distributed across all services

Load Balancing Strategies — Detailed

1. Round Robin Distributes requests sequentially, one by one, in a circular order.

```
Request 1 → Instance 1
Request 2 → Instance 2
Request 3 → Instance 3
Request 4 → Instance 1 (back to first)

Simple and fair, but doesn't consider instance health or current load.
```

2. Weighted Round Robin Instances with higher weight (more powerful servers) get more requests.

```
Instance 1 (weight: 3) → Gets 3 out of every 6 requests
Instance 2 (weight: 2) → Gets 2 out of every 6 requests
Instance 3 (weight: 1) → Gets 1 out of every 6 requests
```

3. Least Connections Routes to the instance with the fewest active connections.

```
Instance 1: 15 active connections
Instance 2: 8 active connections  ← Next request goes here
Instance 3: 22 active connections

Best when requests have varying processing times.
```

4. Random Randomly selects an instance. Simple but less predictable.

5. IP Hash Routes requests from the same client IP to the same instance (session affinity/sticky sessions).

```
Client IP 192.168.1.100 → Always goes to Instance 2
Client IP 192.168.1.200 → Always goes to Instance 1
```

```
Useful for: Caching, session-based applications
Problem: Uneven distribution if some IPs send more traffic
```

Spring Cloud LoadBalancer

Spring Cloud LoadBalancer is the recommended client-side load balancer for Spring Boot (Netflix Ribbon is deprecated).

It integrates with Eureka: when you use `@LoadBalanced` with RestTemplate or use OpenFeign, load balancing happens automatically.

---

14. Saga Pattern

The Fundamental Problem

In a monolith, we have **ACID transactions**:

```sql
BEGIN TRANSACTION;
  INSERT INTO orders (id, product_id, quantity) VALUES (1, 101, 20);
  UPDATE inventory SET stock = stock - 20 WHERE product_id = 101;
  INSERT INTO invoices (order_id, amount) VALUES (1, 5000);
COMMIT;
-- If ANY statement fails, ALL are rolled back. Guaranteed consistency.
```

In microservices, each service has its own database. **You cannot have a single transaction spanning multiple databases.** There is no "distributed COMMIT" that atomically commits to Order DB, Inventory DB, and Billing DB.

Why not use distributed transactions (2PC — Two-Phase Commit)? 2PC exists but is impractical for microservices because:

- It requires all participants to be available during the entire transaction (reduces availability)
- It holds locks across databases for the duration (reduces performance)
- It doesn't scale well
- It creates tight coupling between services

Saga is the alternative: Instead of one big transaction, break it into a **sequence of local transactions**, each with a **compensating action** for rollback.

How Saga Works

Each step in a saga:

1. Performs a local transaction (in its own database)
2. Publishes an event or notifies the next step
3. Has a compensating action that can undo the transaction

```
SAGA: Place Order

Forward actions:                    Compensating actions:
T1: Create Order (PENDING)          C1: Cancel Order
T2: Reserve Stock (-20)             C2: Restore Stock (+20)
T3: Process Payment                 C3: Refund Payment
T4: Generate Invoice                C4: Cancel Invoice
T5: Update Order (CONFIRMED)        C5: (none needed)
```

```
Happy path:
T1 ✅ → T2 ✅ → T3 ✅ → T4 ✅ → T5 ✅ → SAGA COMPLETE

Failure at T3 (payment fails):
T1 ✅ → T2 ✅ → T3 ❌ → C2 (restore stock) → C1 (cancel order) → SAGA FAILED
```

Choreography-Based Saga — In Depth

Services communicate through **events**. Each service listens for events, performs its action, and publishes new events. There is **no central coordinator**.

```
┌──────────────────────────────────────────────────────────────────────┐
│                     CHOREOGRAPHY SAGA FLOW                           │
│                                                                      │
│  Order          [Message        Inventory      [Message      Billing │
│  Service         Broker]        Service         Broker]      Service │
│    │                │              │               │           │    │
│    │── OrderCreated─→              │               │           │    │
│    │                │──OrderCreated→               │           │    │
│    │                │              │               │           │    │
│    │                │              │─StockReserved─→           │    │
│    │                │              │               │─StockRes.→│    │
│    │                │              │               │           │    │
│    │                │              │               │←PaymentOK─│    │
│    │←──PaymentCompleted────────────│               │           │    │
│    │                │              │               │           │    │
│    │ Update Order   │              │               │           │    │
│    │ to CONFIRMED   │              │               │           │    │
│                                                                      │
│  FAILURE CASE (Payment fails):                                       │
│    │                │              │               │←PayFailed─│    │
│    │                │              │←─PayFailed────│           │    │
│    │                │              │               │           │    │
│    │                │              │ Restore Stock │           │    │
│    │                │              │─StockRestored→│           │    │
│    │←──StockRestored───────────────│               │           │    │
│    │                │              │               │           │    │
│    │ Cancel Order   │              │               │           │    │
└──────────────────────────────────────────────────────────────────────┘
```

Advantages of Choreography:

- Loose coupling — services don't know about each other directly
- No single point of failure (no orchestrator)
- Simple for small, straightforward flows

Disadvantages of Choreography:

- Hard to understand the complete flow (logic spread across services)
- Difficult to debug — "what happened to order 123?" requires checking logs of all services
- Risk of cyclic dependencies between services
- Adding a new step requires modifying existing services (to publish/listen for new events)

Orchestration-Based Saga — In Depth

A **central orchestrator** (Saga Orchestrator) directs the entire flow. It tells each service what to do, tracks the progress, and handles failures.

```
┌──────────────────────────────────────────────────────────────────────┐
│                    ORCHESTRATION SAGA FLOW                           │
│                                                                      │
│                    Saga Orchestrator                                 │
│                          │                                           │
│          ┌───────────────┼──────────────────────┐                   │
│          │               │                      │                   │
│          ▼               ▼                      ▼                   │
│    ┌──────────┐    ┌──────────┐           ┌──────────┐             │
│    │  Order   │    │Inventory │           │ Billing  │             │
│    │ Service  │    │ Service  │           │ Service  │             │
│    └──────────┘    └──────────┘           └──────────┘             │
│                                                                      │
│   Step 1: Orchestrator → Order Service: "Create order"              │
│           Order Service → Orchestrator: "Order created (ID: 123)"   │
│                                                                      │
│   Step 2: Orchestrator → Inventory: "Reserve 20 items for order 123"│
│           Inventory → Orchestrator: "Stock reserved"                 │
│                                                                      │
│   Step 3: Orchestrator → Billing: "Charge ₹5000 for order 123"     │
│           Billing → Orchestrator: "Payment successful"               │
│                                                                      │
│   Step 4: Orchestrator → Order Service: "Update order 123 to CONFIRMED"│
│           Order Service → Orchestrator: "Order confirmed"            │
│                                                                      │
│   DONE! Orchestrator marks saga as COMPLETED.                        │
│                                                                      │
│   FAILURE at Step 3 (Payment failed):                                │
│   Orchestrator → Inventory: "Restore stock for order 123"           │
│   Orchestrator → Order Service: "Cancel order 123"                  │
│   Orchestrator marks saga as FAILED with reason.                     │
└──────────────────────────────────────────────────────────────────────┘
```

Advantages of Orchestration:

- Easy to understand — entire flow visible in the orchestrator
- Easy to debug — orchestrator has the complete saga state
- Easy to add new steps — only modify the orchestrator
- Better for complex, multi-step workflows
- Centralized error handling and compensation

Disadvantages of Orchestration:

- Orchestrator can become a single point of failure
- Risk of putting too much logic in the orchestrator (should only coordinate, not contain business logic)
- Slightly more coupling (services know about the orchestrator)

Saga State Machine

The orchestrator typically implements a **state machine** to track saga progress:

```
States:
  STARTED → ORDER_CREATED → STOCK_RESERVED → PAYMENT_COMPLETED → CONFIRMED
                                                  |
                                            PAYMENT_FAILED
                                                  |
                                            STOCK_RESTORED → ORDER_CANCELLED → FAILED

Each state transition triggers the next action or compensation.
```

When to Use What

Criteria	Choreography	Orchestration
Number of services	2-3	4+
Flow complexity	Simple, linear	Complex, branching
Debugging needs	Low	High
Coupling preference	Loose	Acceptable
New steps frequency	Rare	Frequent
Visibility	Distributed (hard)	Centralized (easy)

---

15. CQRS Pattern

The Problem in Detail

In a traditional CRUD application, the same data model is used for both reading and writing:

```
┌──────────────┐
│ Product Table  │
│                │
│  id            │
│  name          │
│  description   │  ← Same model for:
│  price         │     - Admin adding/updating products (WRITE)
│  category_id   │     - Customer searching products (READ)
│  brand_id      │     - Reports on product sales (READ)
│  vehicle_ids   │
│  stock         │
│  created_at    │
│  updated_at    │
└──────────────┘
```

Problems:

1. **Read and Write have different needs:** Writes need normalization (3NF) for data integrity. Reads need denormalization for performance.

2. **Read-heavy applications suffer:** If 95% of operations are reads (customers browsing) and 5% are writes (admin updates), the read queries fight for database resources with write operations.

3. **Complex queries slow down writes:** A customer searches "Bosch oil filter for Swift 2020" — this requires joining products, brands, categories, and vehicle_compatibility tables. This complex query locks resources that writes need.

4. **Single model compromise:** The data model is a compromise between read and write needs, optimized for neither.

CQRS Solution — Detailed

Separate the read model and write model completely:

```
┌──────────────────────────────────────────────────────────┐
│                    WRITE SIDE (Command)                  │
│                                                          │
│  Admin: "Add product: Bosch Oil Filter, ₹250, Swift"     │
│         "Update price to ₹275"                           │
│         "Delete discontinued product"                    │
│                                                          │
│  ┌──────────────────┐                                    │
│  │  Command API     │                                    │
│  │  (POST, PUT,     │                                    │
│  │   DELETE)        │                                    │
│  └──────────────────┘                                    │
│           │                                              │
│           ▼                                              │
│  ┌──────────────────┐                                    │
│  │     MySQL        │  ← Normalized, ACID, consistent    │
│  │ (Write Database) │  ← Optimized for data integrity    │
│  └──────────────────┘                                    │
│           │                                              │
│           │ Sync (via events or Change Data Capture)     │
│           │                                              │
└───────────┼──────────────────────────────────────────────┘
            │
            ▼
┌──────────────────────────────────────────────────────────┐
│                    READ SIDE (Query)                     │
│                                                          │
│  Customer: "Search Bosch oil filter for Swift"           │
│            "Show all brake pads under ₹500"              │
│            "Filter by brand and vehicle"                 │
│                                                          │
│  ┌──────────────────┐                                    │
│  │   Query API      │                                    │
│  │   (GET)          │                                    │
│  └──────────────────┘                                    │
│           │                                              │
│           ▼                                              │
│  ┌──────────────────┐                                    │
│  │ Elasticsearch    │  ← Denormalized, fast search       │
│  │ (Read Database)  │  ← Optimized for query performance │
│  └──────────────────┘                                    │
└──────────────────────────────────────────────────────────┘
```

How Data Syncs Between Write and Read Models

Option 1: Event-Based Sync

```
1. Admin updates product price in MySQL
2. Product Service publishes "ProductPriceUpdated" event
3. Event handler catches the event
4. Updates the denormalized data in Elasticsearch

MySQL (write) ──event──→ Event Handler ──update──→ Elasticsearch (read)
```

Option 2: Change Data Capture (CDC)

```
A tool like Debezium monitors the MySQL transaction log (binlog).
When a row changes, Debezium publishes an event to Kafka.
A consumer reads from Kafka and updates Elasticsearch.

MySQL (binlog) ──→ Debezium ──→ Kafka ──→ Consumer ──→ Elasticsearch
```

Eventual Consistency

CQRS introduces **eventual consistency** — the read model may be slightly behind the write model. When an admin updates a price, there's a brief delay before the customer sees the new price.

```
Time 0: Admin updates price to ₹275 in MySQL (write model)
Time 0: Customer searches, Elasticsearch still shows ₹250 (stale!)
Time 2s: Event processed, Elasticsearch updated to ₹275
Time 2s+: Customer searches, sees ₹275 (consistent now)

The "eventual" delay is typically milliseconds to seconds.
```

Is this acceptable? In most cases, yes. A customer seeing a price that's 2 seconds stale is perfectly fine. But for financial transactions where exact consistency matters, CQRS might not be the right choice (or requires additional safeguards).

Benefits

1. **Independent Scaling:** Scale read side (10 Elasticsearch nodes) and write side (1 MySQL master) separately
2. **Optimized Models:** Write model normalized for integrity, Read model denormalized for speed
3. **Better Performance:** Complex search queries don't affect write operations
4. **Flexibility:** Can use different databases optimized for different access patterns

When to Use and When NOT to Use

Use CQRS when:

- Read operations significantly outnumber writes (>10:1 ratio)
- Read and write models have fundamentally different structures
- You need different databases for different access patterns
- Complex search/filter/aggregation requirements on the read side
- You need to scale reads and writes independently

Don't use CQRS when:

- Simple CRUD application with no complex queries
- Read and write patterns are similar
- Strict real-time consistency is required
- Small dataset that fits comfortably in one database
- Team doesn't have experience with distributed systems (CQRS adds complexity)

---

16. Event Sourcing

Traditional State Storage vs Event Sourcing

Traditional Approach — Store Current State:

```
inventory table:
| product_id | product_name | stock |
|------------|--------------|-------|
| 101        | Oil Filter   | 50    |
| 102        | Brake Pad    | 30    |
```

```
When stock changes: UPDATE inventory SET stock = 50 WHERE product_id = 101;
Previous state is LOST. You only know the current stock is 50.
```

Event Sourcing — Store Events:

```
event_store table:
| event_id | aggregate_id | event_type    | data               | timestamp           |
|----------|-------------|---------------|-------------------|---------------------|
| 1        | 101         | StockAdded    | { quantity: 100 } | 2024-01-01 10:00:00 |
| 2        | 101         | StockSold     | { quantity: 20,   | 2024-01-05 14:30:00 |
|          |             |               |   orderId: 1001 } |                     |
| 3        | 101         | StockSold     | { quantity: 15,   | 2024-01-08 09:15:00 |
|          |             |               |   orderId: 1002 } |                     |
| 4        | 101         | StockReturned | { quantity: 5,    | 2024-01-10 11:00:00 |
|          |             |               |   orderId: 1001,  |                     |
|          |             |               |   reason: "defect"}|                     |
| 5        | 101         | StockSold     | { quantity: 20,   | 2024-01-15 16:45:00 |
|          |             |               |   orderId: 1003 } |                     |

Current stock of product 101 = 100 - 20 - 15 + 5 - 20 = 50
(Calculated by replaying all events)
```

Key Benefits — Explained in Detail

1. Complete Audit Trail Every change is recorded. You know exactly what happened, when, and why.

```
"Why is Oil Filter stock at 50?"
→ Replay events:
  Jan 1: Received 100 from supplier (PO #5001)
  Jan 5: Sold 20 to Sharma Garage (Order #1001)
  Jan 8: Sold 15 to Gupta Motors (Order #1002)
  Jan 10: Sharma returned 5 (defective batch)
  Jan 15: Sold 20 to Yadav Workshop (Order #1003)
```

2. Time Travel (Temporal Query) You can reconstruct the state at any point in time.

```
"What was the stock on January 8th?"
→ Replay events up to Jan 8: 100 - 20 - 15 = 65

"What was the stock before Sharma's return?"
→ Replay events up to event #3: 100 - 20 - 15 = 65
```

3. Event Replay If something went wrong, you can fix the bug and replay all events to get the correct state.

```
Bug: Stock calculation was wrong (wasn't counting returns)
Fix: Fix the calculation logic
Replay: Process all events through the fixed logic
Result: Correct stock numbers without manual correction
```

4. Debugging When a customer complains, you can see the exact sequence of events.

```
"Customer says they returned 5 items but weren't credited"
→ Check events for that order:
  Event #3: StockSold (quantity: 20, order: 1001)
  ...
```

```
  No StockReturned event found for order 1001!
→ The return event was never published — that's the bug.
```

5. Analytics and Insights Historical events provide rich data for business analysis.

```
"Which products have the highest return rate?"
"What's the average time between restocking?"
"Which customers buy the most frequently?"
All answerable from the event store.
```

Snapshots — Performance Optimization

Replaying thousands of events every time you need the current state is slow. **Snapshots** solve this.

```
Events 1-1000: (replayed and summarized)
Snapshot at event 1000: { product_id: 101, stock: 250, timestamp: "2024-06-01" }

To get current state:
1. Load latest snapshot: stock = 250 (at event 1000)
2. Replay only events after 1000: events 1001-1050
3. Current state = snapshot + recent events

Much faster than replaying all 1050 events!
```

Event Sourcing + CQRS

Event Sourcing is often combined with CQRS:

```
WRITE SIDE (Event Sourcing):
  Command → Process → Store Event → Event Store (append-only)

READ SIDE (Materialized View):
  Event Store → Event Handler → Update Read Model → Read Database

┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│   Event Store    │────→│  Event Handler   │────→│  Read Database   │
│ (append-only)    │     │ (processes events)│     │  (materialized   │
│                  │     │                  │     │   view)          │
│ StockAdded 100   │     │ Calculates       │     │ Oil Filter: 50   │
│ StockSold  20    │     │ current state    │     │ Brake Pad: 30    │
│ StockSold  15    │     │ from events      │     │                  │
│ StockRet.   5    │     │                  │     │ (fast reads)     │
│ StockSold  20    │     │                  │     │                  │
└──────────────────┘     └──────────────────┘     └──────────────────┘
```

When to Use Event Sourcing

Use Case	Suitable?	Why
Financial/Banking	✅ Yes	Every transaction must be auditable
Inventory Management	✅ Yes	Stock history and audit trail critical
Order Management	✅ Yes	Order lifecycle tracking important
User Profile CRUD	❌ No	Simple state storage is sufficient

Use Case	Suitable?	Why
Session Management	❌ No	Only current state matters
Real-time Chat	❌ No	Message history, but not event-sourcing level

---

17. Distributed Tracing

The Problem — Why Is It Hard to Debug Microservices?

In a monolith, a request goes through functions in the same process. You can set a breakpoint, look at one log file, and trace the flow easily.

In microservices, a single user request can touch 5-10 services, each with its own logs, running on different servers. If the request is slow or fails, finding the root cause is extremely difficult.

```
Customer: "My order is taking too long!"

The request path:
Client → API Gateway → Auth Service → Order Service → Product Service
                                                     → Inventory Service
                                                     → Payment Gateway
                                                     → Billing Service
                                                     → Notification Service

Question: Which of these 8 services is causing the delay?
Answer: Without distributed tracing, you'd have to check logs of all 8 services
        and try to correlate them by timestamp. Manual and error-prone.
```

How Distributed Tracing Works

Two key concepts:

Trace: The entire journey of a request through the system. One trace per user request.

Span: A single unit of work within a service. Each service creates one or more spans.

```
Trace ID: abc-123 (unique ID for the entire request)
│
├── Span 1: API Gateway         (Span ID: s1, Parent: none)     [10ms]
│   │
│   └── Span 2: Order Service   (Span ID: s2, Parent: s1)       [100ms]
│       │
│       ├── Span 3: Product Svc (Span ID: s3, Parent: s2)       [500ms] ← SLOW!
│       │
│       ├── Span 4: Inventory   (Span ID: s4, Parent: s2)       [20ms]
│       │
│       └── Span 5: Billing Svc (Span ID: s5, Parent: s2)       [25ms]

All spans share the same Trace ID: abc-123
Each span has its own Span ID and a reference to its Parent Span
```

How the Trace ID propagates:

```
1. Client sends request to API Gateway
2. Gateway generates Trace ID: "abc-123" and Span ID: "s1"
3. Gateway adds headers to downstream call:
   X-B3-TraceId: abc-123
   X-B3-SpanId: s1

4. Order Service receives request, reads Trace ID from header
   Creates new Span: "s2" with Parent: "s1"

5. Order Service calls Product Service with headers:
   X-B3-TraceId: abc-123     (same trace!)
   X-B3-SpanId: s2           (parent for next span)

6. Product Service creates Span: "s3" with Parent: "s2"

7. All services report their spans to Zipkin
8. Zipkin stitches them together using Trace ID
```

Zipkin Dashboard Visualization

```
Trace: abc-123 (Total: 585ms)

Service          Timeline
──────────────────────────────────────────────────────────────────→ time
API Gateway      [===]                                             10ms
Order Service      [=========================]                   100ms
Product Service       [==========================================] 500ms
Inventory Service     [====]                                      20ms
Billing Service              [=====]                              25ms

Immediately visible: Product Service is the bottleneck at 500ms!
```

What Information Does a Span Contain?

Field	Description	Example
Trace ID	Unique ID for entire request	abc-123
Span ID	Unique ID for this span	span-42
Parent Span ID	ID of the calling span	span-12
Operation Name	What the span represents	"GET /products/123"
Service Name	Which service created this span	"product-service"
Start Timestamp	When the span started	2024-01-15T10:30:00.123Z
Duration	How long the span took	500ms
Tags	Key-value metadata	http.method=GET, http.status=200
Logs/Events	Timestamped events within span	"Cache miss", "DB query executed"

Tools

Tool	Description
Zipkin	Open-source, created by Twitter. Simple UI. Good for Spring Boot.

Tool	Description
Jaeger	Open-source, created by Uber. More features than Zipkin. Kubernetes-native.
OpenTelemetry	Vendor-neutral standard for tracing, metrics, and logs. Becoming the industry standard.
Micrometer Tracing	Spring Boot integration that bridges to Zipkin/Jaeger/OpenTelemetry.
AWS X-Ray	AWS-managed tracing service.

Spring Boot Integration

In Spring Boot 3+, you use Micrometer Tracing (previously Spring Cloud Sleuth):

```
# application.yml
management:
  tracing:
    sampling:
      probability: 1.0    # Sample 100% of requests (use 0.1 for 10% in production)
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
```

Micrometer automatically:

- Generates Trace ID and Span ID
- Propagates headers across service calls (RestTemplate, WebClient, OpenFeign)
- Reports spans to Zipkin
- Adds Trace ID to log entries (MDC)

---

18. Centralized Logging

The Problem

With multiple microservices, each generating its own logs:

```
Product Service (Server 1):  /var/log/product-service.log
Order Service (Server 2):    /var/log/order-service.log
Billing Service (Server 3):  /var/log/billing-service.log
Gateway (Server 4):          /var/log/gateway.log
Eureka (Server 5):           /var/log/eureka.log

5 services on 5 servers = 5 different places to check.
In production with replicas: 5 services × 3 instances = 15 log files!
```

Debugging without centralized logging:

```
1. SSH into Server 1, grep for error in product-service.log
2. SSH into Server 2, grep for error in order-service.log
3. SSH into Server 3, grep for error in billing-service.log
4. Try to correlate timestamps manually
5. Hope the server clocks are synchronized
6. Miss the error because it was on a different instance
```

```
Time spent: 30-60 minutes for what should be a 2-minute investigation.
```

Solution: ELK Stack

The ELK Stack (Elasticsearch + Logstash + Kibana) is the most popular centralized logging solution.

```
┌──────────┐  ┌──────────┐  ┌──────────┐
│ Service 1│  │ Service 2│  │ Service 3│
│  logs    │  │  logs    │  │  logs    │
└──────────┘  └──────────┘  └──────────┘
      │             │             │
      └─────────────┴─────────────┘
                    │
                    ▼
            ┌──────────────┐
            │   Logstash   │    COLLECT & PROCESS
            │              │    - Parse log format
            │              │    - Extract fields
            │              │    - Transform data
            │              │    - Filter noise
            └──────────────┘
                    │
                    ▼
            ┌──────────────┐
            │Elasticsearch │    STORE & INDEX
            │              │    - Full-text search
            │              │    - Fast queries
            │              │    - Aggregations
            └──────────────┘
                    │
                    ▼
            ┌──────────────┐
            │    Kibana    │    VISUALIZE & ANALYZE
            │              │    - Dashboards
            │              │    - Search interface
            │              │    - Alerts
            │              │    - Charts & graphs
            └──────────────┘
```

Each component explained:

Elasticsearch:

- Distributed search and analytics engine
- Stores logs as JSON documents
- Provides extremely fast full-text search
- Supports complex queries and aggregations
- Horizontally scalable

Logstash:

- Data collection and processing pipeline
- Collects logs from multiple sources
- Parses unstructured logs into structured data
- Filters out noise (debug logs in production)
- Routes logs to Elasticsearch

Kibana:

- Web-based visualization tool
- Provides a dashboard to search and explore logs
- Create visualizations (charts, graphs, maps)
- Set up alerts (notify when error rate spikes)
- Real-time log tailing

Best Practices for Microservices Logging

1. Structured Logging (JSON format)

```
Unstructured (bad):
2024-01-15 10:30:00 ERROR OrderService - Failed to process order 123 for customer 456

Structured (good):
{
  "timestamp": "2024-01-15T10:30:00.123Z",
  "level": "ERROR",
  "service": "order-service",
  "traceId": "abc-123",
  "spanId": "span-42",
  "message": "Failed to process order",
  "orderId": 123,
  "customerId": 456,
  "error": "PaymentDeclinedException",
  "duration_ms": 1500
}
```

2. Include Trace ID in Every Log Entry This allows you to search for a single Trace ID and see logs from ALL services related to that request.

```
Search in Kibana: traceId = "abc-123"
Results:
  10:30:00.100 [gateway]          Request received: GET /orders/123
  10:30:00.110 [order-service]    Processing order 123
  10:30:00.150 [product-service]  Fetching product for order 123
  10:30:00.200 [product-service]  Product found: Oil Filter
  10:30:00.250 [billing-service]  Payment processing for order 123
  10:30:01.600 [billing-service]  ERROR: Payment declined
  10:30:01.610 [order-service]    ERROR: Failed to process order 123
```

3. Use Appropriate Log Levels

```
ERROR   → Something failed, needs attention       (payment declined)
WARN    → Something unexpected, but handled        (retry succeeded on 2nd attempt)
INFO    → Normal business events                   (order placed, payment received)
DEBUG   → Detailed technical information           (SQL query, cache hit/miss)
TRACE   → Very detailed, step-by-step execution    (method entry/exit)
```

4. Don't Log Sensitive Data Never log passwords, credit card numbers, personal data (GDPR), API keys, or tokens.

Alternative: EFK Stack

Component	ELK	EFK
Collector	**Logstash** (heavy, more features)	**Fluentd/Fluent Bit** (lightweight, K8s native)

Component	ELK	EFK
Storage	Elasticsearch	Elasticsearch
Visualization	Kibana	Kibana

EFK is preferred in Kubernetes environments because Fluent Bit is lightweight and has native K8s integration.

---

19. 12-Factor App Methodology

Background

The 12-Factor App methodology was created by developers at **Heroku** (a cloud platform). It provides 12 best practices for building applications that are:

- **Portable** across execution environments
- **Scalable** without significant architecture changes
- **Suitable for deployment** on modern cloud platforms
- **Minimizing divergence** between development and production

These principles align perfectly with microservices architecture.

The 12 Factors — Detailed

Factor 1: Codebase — "One codebase tracked in revision control, many deploys"

One service = one Git repository. The same codebase is deployed to different environments (dev, staging, prod). Different environments should never have different code — only different configuration.

```
product-service (Git repo)
  ├── Deployed to DEV   (latest commits)
  ├── Deployed to STAGING (release candidate)
  └── Deployed to PROD  (stable release)

Same code, different config (DB URLs, API keys, etc.)
```

Violation: Two services sharing a codebase, or copying code between repos.

Factor 2: Dependencies — "Explicitly declare and isolate dependencies"

All dependencies must be declared in a manifest file. The application should never rely on implicitly available system-wide packages.

```
Java/Spring Boot: pom.xml or build.gradle
  - Every library explicitly listed with version
  - Running "mvn install" or "gradle build" gets everything needed

Node.js: package.json + package-lock.json
Python: requirements.txt or Pipfile
```

Violation: Assuming a system library is installed, or depending on a global tool that isn't in the dependency manifest.

Factor 3: Config — "Store config in the environment"

This is one of the most important factors for microservices.

Configuration that varies between environments (database URLs, API keys, credentials, service URLs) must NOT be stored in code. It should come from environment variables or a Config Server.

```
❌ Wrong:
public class DatabaseConfig {
    private String url = "jdbc:mysql://localhost:3306/productdb";  // Hardcoded!
    private String password = "secret123";  // SECRET IN CODE!
}

✅ Correct:
public class DatabaseConfig {
    @Value("${spring.datasource.url}")      // From Config Server or env var
    private String url;

    @Value("${spring.datasource.password}") // From Config Server (encrypted)
    private String password;
}
```

Test: Could you open-source your codebase right now without exposing any credentials? If yes, your config is properly externalized.

Factor 4: Backing Services — "Treat backing services as attached resources"

Databases, message queues, caches, email services, and other external services should be treated as **attached resources** that can be swapped without code changes.

```
Switching from local MySQL to Amazon RDS should only require a config change:

# Local development
spring.datasource.url=jdbc:mysql://localhost:3306/productdb

# Production (just change the URL)
spring.datasource.url=jdbc:mysql://rds.amazonaws.com:3306/productdb

No code changes needed!
```

Factor 5: Build, Release, Run — "Strictly separate build and run stages"

```
BUILD stage:
  Code + Dependencies → Compiled artifact (JAR) or Docker image
  mvn clean package → product-service-1.0.jar

RELEASE stage:
  Artifact + Environment Config → Ready-to-run release
  Docker image + environment variables for PROD

RUN stage:
  Execute the release
  java -jar product-service-1.0.jar (with PROD config)
```

Each stage is strictly separated. You should never modify code at runtime.

Factor 6: Processes — "Execute the app as one or more stateless processes"

This is critical for microservices.

Services must be **stateless**. They should not store any user data, sessions, or state in local memory or filesystem. All persistent data must be stored in a backing service (database, Redis, etc.).

```
❌ Wrong: Storing user session in application memory
// If this instance crashes, session is lost!
// If load balancer routes to a different instance, session doesn't exist!
Map<String, UserSession> sessions = new HashMap<>();

✅ Correct: Storing session in Redis (external store)
// Any instance can access the session
// Instance crash doesn't affect sessions
redisTemplate.opsForValue().set("session:" + sessionId, userSession);
```

Why stateless? Because instances can be added, removed, or restarted at any time. If state is in memory, it's lost on restart and not available on other instances.

Factor 7: Port Binding — "Export services via port binding"

Each service is **self-contained** and binds to a port to serve requests. It doesn't rely on an external web server (like Tomcat installed separately).

Spring Boot does this by default — it embeds Tomcat and binds to a port:

```
Product Service  → http://localhost:8081
Order Service    → http://localhost:8082
Billing Service  → http://localhost:8083
```

Factor 8: Concurrency — "Scale out via the process model"

Scale by running more instances (horizontal scaling), not by making one instance bigger (vertical scaling).

```
❌ Vertical Scaling:
One instance with 64GB RAM, 32 CPUs
- Expensive hardware
- Single point of failure
- Has an upper limit

✅ Horizontal Scaling:
Four instances with 4GB RAM, 4 CPUs each
- Commodity hardware
- Fault tolerant (one crashes, three remain)
- Can keep adding instances
```

Factor 9: Disposability — "Maximize robustness with fast startup and graceful shutdown"

Services should:

- **Start quickly** (seconds, not minutes) — important for auto-scaling
- **Shut down gracefully** — finish processing current requests, release resources, then exit
- **Be resilient to sudden death** — handle crashes without data corruption

```
Graceful Shutdown:
1. Receive SIGTERM signal
2. Stop accepting new requests
3. Finish processing in-flight requests (with timeout)
4. Close database connections
```

```
5. Exit

Spring Boot supports this with:
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

Factor 10: Dev/Prod Parity — "Keep development, staging, and production as similar as possible"

Minimize the gap between environments:

```
❌ Different tools per environment:
Dev:  H2 in-memory database, no message queue
Prod: MySQL, RabbitMQ

✅ Same tools, different instances:
Dev:  MySQL (local Docker), RabbitMQ (local Docker)
Prod: MySQL (RDS), RabbitMQ (Amazon MQ)
```

Docker and Docker Compose make this easy — run the same stack locally as in production.

Factor 11: Logs — "Treat logs as event streams"

Services should write logs to **stdout** (standard output). They should NOT manage log files, log rotation, or log storage. External tools handle that.

```
Service writes to stdout:
  System.out.println("Order 123 created");
  logger.info("Order {} created", orderId);

External tool (Logstash/Fluentd) captures stdout and sends to Elasticsearch.
```

Factor 12: Admin Processes — "Run admin/management tasks as one-off processes"

Tasks like database migrations, data backups, and one-time scripts should run as separate processes, not as part of the application startup.

```
❌ Wrong: Running database migration on every app startup
   (Multiple instances would run it simultaneously — chaos!)

✅ Correct: Running migration as a separate job
   flyway migrate (run once, before deploying new version)

In Kubernetes: Use a Job or Init Container for migrations
```

---

20. Security in Microservices

Security Challenges Specific to Microservices

1. **Larger attack surface:** More services = more endpoints to protect
2. **Network communication:** Services communicate over the network, which can be intercepted
3. **Multiple authentication points:** Each service needs to verify the caller's identity
4. **Secret management:** Credentials spread across many services

5. **Service-to-service trust:** How does Service A know that the caller is really Service B?

Authentication & Authorization — In Depth

Authentication: "Who are you?" (Verify identity) **Authorization:** "What can you do?" (Verify permissions)

OAuth2 + JWT Flow

```
┌──────────┐      ┌──────────────┐      ┌────────────┐      ┌──────────┐
│  Client  │      │  Auth Server  │      │ API Gateway │      │Microservice│
│(Browser) │      │  (Keycloak)   │      │            │      │          │
└────┬─────┘      └──────┬───────┘      └─────┬──────┘      └────┬─────┘
     │                   │                    │                  │
     │ 1. Login (user/pass)                   │                  │
     │──────────────────→│                    │                  │
     │                   │                    │                  │
     │ 2. Return JWT Token                    │                  │
     │←──────────────────│                    │                  │
     │                   │                    │                  │
     │ 3. API Call + JWT Token                │                  │
     │────────────────────────────────────────→│                  │
     │                   │                    │                  │
     │                   │      4. Validate JWT Token            │
     │                   │        (check signature,             │
     │                   │         expiry, claims)              │
     │                   │                    │                  │
     │                   │      5. If valid, forward request    │
     │                   │                    │──────────────────→│
     │                   │                    │                  │
     │                   │      6. Response   │                  │
     │                   │                    │←─────────────────│
     │                   │                    │                  │
     │ 7. Response to client                  │                  │
     │←───────────────────────────────────────│                  │
```

JWT Token Structure:

```
Header: { "alg": "RS256", "typ": "JWT" }
Payload: {
  "sub": "user123",
  "name": "Vidit",
  "roles": ["ADMIN", "USER"],
  "iat": 1705299600,
  "exp": 1705303200
}
Signature: RSASHA256(header + payload, private_key)

The token is self-contained — it carries user identity and roles.
The API Gateway validates the signature without calling the Auth Server.
```

Why JWT at the Gateway?

- Validate once at the gateway, not at every service
- Services trust the gateway — if the request reached them, it's already authenticated
- Gateway can add user info to request headers for downstream services

Service-to-Service Security

Problem: How does Product Service know the call is coming from Order Service and not from a malicious actor?

Solution 1: Mutual TLS (mTLS) Both client and server present certificates to verify each other's identity.

```
Order Service → presents its certificate → Product Service
Product Service → verifies Order Service's certificate
Product Service → presents its certificate → Order Service
Order Service → verifies Product Service's certificate

Both sides verified → Communication is encrypted and trusted
```

Solution 2: Service Mesh (Istio) Istio automatically handles mTLS between all services, without any code changes. It injects a sidecar proxy (Envoy) alongside each service that handles encryption and authentication.

Secret Management

Problem: Database passwords, API keys, encryption keys are needed by multiple services. Where to store them securely?

```
❌ In code: password = "secret123"
❌ In config files: committed to Git, visible to everyone
❌ In environment variables: better, but still visible in process listings

✅ HashiCorp Vault: Centralized secret management
   - Secrets stored encrypted
   - Access controlled by policies
   - Secrets can be rotated automatically
   - Audit trail of who accessed what
```

Security Checklist for Microservices

Layer	Mechanism
External Access	API Gateway + JWT/OAuth2 authentication
Rate Limiting	API Gateway limits requests per client
Service-to-Service	Mutual TLS or Service Mesh (Istio)
Secrets	HashiCorp Vault or Kubernetes Secrets
Data in Transit	TLS/HTTPS for all communication
Data at Rest	Database encryption
Input Validation	Validate all input at API boundaries
Logging	Audit log of all security events
Dependency Scanning	Scan for vulnerable dependencies (OWASP)

---

21. Docker & Containerization

The Problem Docker Solves

"It works on my machine!" — the most common developer excuse.

```
Developer's machine:
- Java 17
- MySQL 8.0
- Ubuntu 22.04
- Environment variables set correctly
→ Application works! ✅

Production server:
- Java 11 (different version!)
- MySQL 5.7 (different version!)
- CentOS 8 (different OS!)
- Environment variables missing
→ Application fails! ❌
```

What is Docker?

Docker packages an application with **all its dependencies** (runtime, libraries, config) into a standardized unit called a **container**. The container runs the same way on any machine that has Docker installed.

Key Concepts — Detailed

Docker Image: A read-only template that contains the application code, runtime, libraries, and dependencies. Think of it as a **blueprint** or **class** in OOP.

```
Image: product-service:1.0
Contains:
  - OpenJDK 17 (Java runtime)
  - product-service.jar (your application)
  - application.yml (default config)
  - All Spring Boot dependencies
```

Docker Container: A running instance of an image. Think of it as an **object** created from a class. You can create multiple containers from the same image.

```
Image: product-service:1.0
  → Container 1: Running on port 8081
  → Container 2: Running on port 8082
  → Container 3: Running on port 8083
```

Dockerfile: A text file with instructions to build a Docker image.

```
# Start with a base image that has Java
FROM openjdk:17-jdk-slim

# Set working directory inside the container
WORKDIR /app

# Copy the JAR file into the container
COPY target/product-service-1.0.jar app.jar

# Expose the port the service runs on
EXPOSE 8081

# Command to run when container starts
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Multi-Stage Build (Production Best Practice):

```dockerfile
# Stage 1: Build
FROM maven:3.9-openjdk-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run (smaller image, no build tools)
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY --from=build /app/target/product-service-1.0.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Docker Compose: A tool for defining and running multi-container applications. Essential for microservices.

```yaml
version: '3.8'
services:
  # Infrastructure
  eureka-server:
    build: ./eureka-server
    ports:
      - "8761:8761"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8761/actuator/health"]
      interval: 10s
      retries: 5

  config-server:
    build: ./config-server
    ports:
      - "8888:8888"
    depends_on:
      eureka-server:
        condition: service_healthy

  api-gateway:
    build: ./api-gateway
    ports:
      - "8080:8080"
    depends_on:
      - eureka-server
      - config-server

  # Business Services
  product-service:
    build: ./product-service
    ports:
      - "8081:8081"
    depends_on:
      - eureka-server
      - config-server
      - product-db
    environment:
      - SPRING_DATASOURCE_URL=jdbc:mysql://product-db:3306/product_db

  order-service:
    build: ./order-service
    ports:
      - "8082:8082"
    depends_on:
      - eureka-server
      - config-server
      - order-db

  # Databases
  product-db:
```

```
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: product_db
    volumes:
      - product-data:/var/lib/mysql

  order-db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: order_db
    volumes:
      - order-data:/var/lib/mysql

volumes:
  product-data:
  order-data:
```

Start everything with one command:

```
docker-compose up -d
```

Docker vs Virtual Machines

Feature	Docker Container	Virtual Machine
Size	MBs (lightweight)	GBs (heavy)
Startup	Seconds	Minutes
OS	Shares host OS kernel	Full OS per VM
Isolation	Process-level	Hardware-level
Performance	Near-native	Overhead from hypervisor
Resource Usage	Low	High
Use Case	Microservices	Legacy apps, different OS needs

---

22. Kubernetes & Orchestration

Why Do We Need Kubernetes?

Docker is great for running containers on a single machine. But in production:

- You have **hundreds of containers** across **multiple servers**
- Containers crash and need to be **restarted**
- Traffic changes and containers need to be **scaled**
- New versions need to be deployed with **zero downtime**
- Containers need to **find each other** across servers

Managing this manually is impossible. **Kubernetes automates all of this.**

What is Kubernetes?

Kubernetes (K8s) is an open-source **container orchestration platform**, originally designed by Google, now maintained by the Cloud Native Computing Foundation (CNCF).

Key Concepts — Detailed

Pod: The smallest deployable unit. A pod contains one or more containers that share network and storage. Usually, one pod = one container.

```
Pod: product-service-pod
  └── Container: product-service:1.0
      Listening on port 8081
```

Deployment: Defines the desired state for your pods — how many replicas, which image to use, update strategy.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
spec:
  replicas: 3    # Run 3 instances
  selector:
    matchLabels:
      app: product-service
  template:
    spec:
      containers:
        - name: product-service
          image: product-service:1.0
          ports:
            - containerPort: 8081
```

Service: A stable network endpoint that routes traffic to pods. Pods have dynamic IPs (they change on restart), but a Service has a stable IP/DNS name.

```
Service: product-service (ClusterIP: 10.0.0.15)
  → Pod 1 (10.0.1.5:8081)
  → Pod 2 (10.0.1.6:8081)
  → Pod 3 (10.0.1.7:8081)

Other services call: http://product-service:8081
The Service load-balances across the 3 pods.
```

Ingress: Routes external traffic into the cluster. Acts like an API Gateway.

```
External: https://api.example.com/products → Ingress → Product Service
External: https://api.example.com/orders   → Ingress → Order Service
```

ConfigMap & Secret:

- **ConfigMap:** Stores non-sensitive configuration (similar to Config Server)
- **Secret:** Stores sensitive data (passwords, tokens) — base64 encoded

Namespace: Logical isolation within a cluster. Useful for separating environments.

```
Namespace: dev     → Dev services
Namespace: staging→ Staging services
Namespace: prod    → Production services
```

Key Features

1. Auto-Scaling (Horizontal Pod Autoscaler)

```
Rule: If CPU usage > 70%, add more pods (max 10)
      If CPU usage < 30%, remove pods (min 2)

Normal traffic:    2 pods running
Black Friday sale: Auto-scaled to 8 pods
After sale:        Scaled back to 2 pods
```

2. Self-Healing

```
Pod crashes (OOM, application error):
→ Kubernetes detects the crash
→ Automatically restarts the pod
→ If it keeps crashing, applies exponential backoff
→ Alerts are triggered for investigation
```

3. Rolling Updates (Zero Downtime Deployment)

```
Updating product-service from v1.0 to v2.0:

Step 1: 3 pods running v1.0
Step 2: Start 1 pod with v2.0, stop 1 pod with v1.0
        [v1.0] [v1.0] [v2.0]
Step 3: Start 1 more v2.0, stop 1 more v1.0
        [v1.0] [v2.0] [v2.0]
Step 4: Start last v2.0, stop last v1.0
        [v2.0] [v2.0] [v2.0]

Users experienced ZERO downtime during the update!

If v2.0 has a bug:
→ Automatic rollback to v1.0
```

4. Service Discovery (Built-in) Kubernetes has built-in service discovery — no need for Eureka!

```
Product Service pod can call Order Service by name:
http://order-service:8082/api/orders

Kubernetes DNS resolves "order-service" to the correct pod IPs.
```

Docker Compose vs Kubernetes

Feature	Docker Compose	Kubernetes
Scope	Single machine	Cluster of machines

Feature	Docker Compose	Kubernetes
Auto-scaling	No	Yes
Self-healing	Limited	Yes
Rolling updates	No	Yes
Service discovery	Basic (DNS)	Advanced (built-in)
Load balancing	Basic	Advanced
Use case	Development, small deployments	Production, large-scale

---

23. Spring Cloud Ecosystem

Complete Component Map

```
┌──────────────────────────────────────────────────────────────────┐
│                    SPRING CLOUD ECOSYSTEM                        │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                      CLIENT LAYER                        │   │
│  │  Mobile App / Web Browser / External API                 │   │
│  └──────────────────────────┬───────────────────────────────┘   │
│                             │                                    │
│  ┌──────────────────────────▼───────────────────────────────┐   │
│  │              SPRING CLOUD GATEWAY                        │   │
│  │  • Request routing (path-based)                          │   │
│  │  • JWT authentication filter                             │   │
│  │  • Rate limiting filter                                  │   │
│  │  • Circuit breaker filter                                │   │
│  │  • Load balancing (via Eureka)                           │   │
│  └──────────────────────────┬───────────────────────────────┘   │
│                             │                                    │
│           ┌─────────────────┼─────────────────┐                  │
│           ▼                 ▼                 ▼                  │
│  ┌────────────────┐  ┌────────────────┐  ┌────────────────┐     │
│  │   Product      │  │    Order       │  │   Billing      │     │
│  │   Service      │  │   Service      │  │   Service      │     │
│  │                │  │                │  │                │     │
│  │ • OpenFeign    │  │ • OpenFeign    │  │ • OpenFeign    │     │
│  │   (REST        │  │   (calls       │  │                │     │
│  │    client)     │  │   other svc)   │  │                │     │
│  │ • Resilience   │  │ • Resilience   │  │ • Resilience   │     │
│  │   4j           │  │   4j           │  │   4j           │     │
│  │ • Micrometer   │  │ • Micrometer   │  │ • Micrometer   │     │
│  │   Tracing      │  │   Tracing      │  │   Tracing      │     │
│  └────────┬───────┘  └────────┬───────┘  └────────┬───────┘     │
│           │                  │                  │                │
│       [MySQL]            [MySQL]            [MySQL]              │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                INFRASTRUCTURE SERVICES                   │   │
│  │                                                          │   │
│  │  ┌────────────────┐  ┌────────────────┐  ┌────────────────┐ │
│  │  │ Config Server  │  │ Eureka Server  │  │    Zipkin      │ │
│  │  │ (Git-backed    │  │ (Service       │  │ (Distributed   │ │
│  │  │  central       │  │  Registry)     │  │  Tracing)      │ │
│  │  │  config)       │  │                │  │                │ │
│  │  └────────────────┘  └────────────────┘  └────────────────┘ │
│  │                                                          │   │
│  │  ┌────────────────┐  ┌────────────────┐                  │   │
```

```
│  │  │  RabbitMQ /    │  │    ELK Stack   │                    │   │
│  │  │  Kafka         │  │  (Centralized  │                    │   │
│  │  │ (Messaging)    │  │   Logging)     │                    │   │
│  │  └────────────────┘  └────────────────┘                    │   │
│  └────────────────────────────────────────────────────────────┘   │
└────────────────────────────────────────────────────────────────────┘
```

Complete Component Reference

Concern	Spring Cloud Tool	Purpose
Centralized Config	**Spring Cloud Config**	Externalized config backed by Git
Service Discovery	**Netflix Eureka**	Service registration and discovery
API Gateway	**Spring Cloud Gateway**	Routing, filtering, security
Client-Side Load Balancing	**Spring Cloud LoadBalancer**	Distribute calls across instances
Circuit Breaker	**Resilience4j**	Fault tolerance (CB, retry, timeout, bulkhead)
Declarative REST Client	**OpenFeign**	Type-safe HTTP client for service calls
Distributed Tracing	**Micrometer Tracing + Zipkin**	Request tracing across services
Messaging	**Spring Cloud Stream**	Abstraction over RabbitMQ/Kafka
Security	**Spring Security + OAuth2**	Authentication and authorization
Config Refresh	**Spring Cloud Bus**	Broadcast config changes to all services

Service Startup Order

```
1. Config Server    → Must start first (all services need config)
     ↓
2. Eureka Server    → Fetches config from Config Server
     ↓
3. API Gateway      → Fetches config, registers with Eureka
     ↓
4. Business Services→ Fetch config, register with Eureka
     ↓
5. Zipkin/ELK      → Can start anytime (independent)
```

---

24. Additional Design Patterns

Strangler Fig Pattern

Purpose: Gradually migrate from monolith to microservices.

Named after the strangler fig tree that grows around a host tree, eventually replacing it.

```
Phase 1: All traffic → Monolith
┌──────────────────────────────┐
│           MONOLITH           │
│  Product | Order | Billing   |
```

```
└──────────────────────────────┘

Phase 2: New features as microservices, gradually extract existing ones
┌──────────────────────────────┐      ┌──────────────────┐
│           MONOLITH           │      │ Product Service  │
│  (Order | Billing)           │      │ (extracted)      │
└──────────────────────────────┘      └──────────────────┘

Phase 3: Continue extracting
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│ Product Svc  │  │  Order Svc   │  │ Billing Svc  │  │  MONOLITH    │
│ (extracted)  │  │ (extracted)  │  │ (extracted)  │  │  (empty!)    │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘

Phase 4: Monolith fully replaced and decommissioned
```

How it works:

1. Put a **facade (API Gateway)** in front of the monolith
2. New features are built as microservices behind the facade
3. Existing features are gradually extracted from monolith into microservices
4. The facade routes traffic to the appropriate destination (monolith or microservice)
5. Eventually, the monolith has no functionality left and is decommissioned

Bulkhead Pattern

Purpose: Isolate resources so that a failure in one component doesn't bring down everything.

Named after the compartments (bulkheads) in a ship — if one compartment floods, the others remain intact.

```
Without Bulkhead:
Order Service has ONE thread pool (100 threads) for ALL outgoing calls:
  → Product Service calls (using 90 threads — it's slow!)
  → Billing Service calls (only 10 threads left — starving!)
  → Customer Service calls (0 threads — BLOCKED!)

With Bulkhead:
Order Service has SEPARATE thread pools:
  Thread Pool A (40 threads) → Product Service calls
  Thread Pool B (30 threads) → Billing Service calls
  Thread Pool C (30 threads) → Customer Service calls

Product Service is slow → Pool A exhausted
But Pool B and C are fine → Billing and Customer calls still work!
```

Sidecar Pattern

Purpose: Deploy supporting functionality alongside a service without modifying the service itself.

```
┌──────────────────────────────────┐
│           POD                    │
│                                  │
│  ┌──────────────┐  ┌──────────┐ │
│  │ Product      │  │ Sidecar  │ │
│  │ Service      │  │ (Envoy   │ │
│  │ (main app)   │←→│  Proxy)  │ │
│  │              │  │          │ │
│  │              │  │ • mTLS   │ │
│  │              │  │ • Tracing│ │
│  │              │  │ • Metrics│ │
```

```
│  └──────────────┘   └──────────┘  │
└──────────────────────────────────┘
```

The sidecar handles cross-cutting concerns (security, logging, monitoring) without the main service needing any changes. Used extensively in **service meshes** like Istio.

Backends for Frontends (BFF) Pattern

Purpose: Create separate API backends optimized for different frontend clients.

```
Mobile App → [Mobile BFF] → Microservices
             (Returns compact JSON, fewer fields,
              pagination optimized for mobile)

Web App   → [Web BFF]    → Microservices
             (Returns full JSON, more fields,
              optimized for desktop screens)

Third-party→ [Public API] → Microservices
             (Stable, versioned API with rate limits)
```

API Composition Pattern

Purpose: Aggregate data from multiple services into a single response.

```
Client needs: Order details with product info and customer info

API Composer (could be in Gateway or separate service):
  1. Call Order Service → get order data
  2. Call Product Service → get product details
  3. Call Customer Service → get customer details
  4. Combine into single response
  5. Return to client

Client gets one response instead of making three calls.
```

---

25. Interview Questions & In-Depth Answers

Q1: What are Microservices?

Answer: Microservices is an architectural style where a large application is structured as a collection of small, autonomous services. Each service is built around a specific business capability, owns its data, and communicates with other services via well-defined APIs, typically REST or messaging. Each service can be independently developed, deployed, and scaled. The architecture promotes loose coupling and high cohesion, enabling teams to work independently and release features faster. Key characteristics include database per service, decentralized governance, design for failure, and infrastructure automation. The approach was popularized by Martin Fowler and James Lewis, and is used by companies like Netflix, Amazon, and Uber for building large-scale, resilient systems.

Q2: What is the difference between Monolith and Microservices?

Answer: A monolith is a single deployable unit where all modules share one codebase, one process, and one database. Communication between modules happens via in-process function calls. It's simpler to develop and test initially, but becomes difficult to scale and maintain as it grows. A single bug can crash the entire application, and the entire app must be redeployed for any change.

Microservices decompose the application into independent services, each with its own codebase, database, and deployment. Communication happens over the network via REST or messaging. Each service can be independently deployed, scaled, and even written in different languages. However, it introduces distributed system complexity — network failures, data consistency challenges, and operational overhead. The choice depends on the application's complexity, team size, and scaling needs. Martin Fowler advises: start with a monolith and extract microservices when the need arises.

Q3: What are the advantages and disadvantages of Microservices?

Answer: The main advantages are: independent deployment (deploy one service without affecting others), granular scalability (scale only what needs scaling), fault isolation (one service failure doesn't crash the system), technology freedom (use the best tool for each service), team autonomy (small teams own services end-to-end), and faster development through parallel work.

The main disadvantages are: distributed system complexity (network failures, latency), data consistency challenges (no ACID transactions across services), testing complexity (integration testing requires multiple services), debugging difficulty (requests span multiple services), operational overhead (each service needs its own CI/CD, monitoring, logging), and increased infrastructure cost.

It's important to note that microservices are not a silver bullet. For small applications with small teams, a monolith is often the better choice. The benefits of microservices only outweigh the costs when the application and team are large enough.

Q4: How do you define service boundaries?

Answer: I use Domain-Driven Design (DDD) to define service boundaries. The process involves identifying the business domain, breaking it into subdomains, and defining bounded contexts. Each bounded context typically maps to one microservice.

Key principles for defining boundaries: each service should have a single responsibility (one reason to change), own its data completely, be independently deployable, and represent a clear business capability. A good test is whether you can explain what the service does in one sentence.

Warning signs of wrong boundaries include: two services that always need to change together (maybe they should be one service), excessive inter-service communication (boundaries might be wrong), and one service that changes for multiple unrelated reasons (maybe it should be split).

Q5: Explain Service Discovery with Eureka.

Answer: Service Discovery solves the problem of services finding each other in a dynamic environment where instances are created, destroyed, and moved constantly. Netflix Eureka provides client-side service discovery.

When a service starts, it registers itself with Eureka Server, providing its name and network address. It then sends heartbeats every 30 seconds to indicate it's alive. If Eureka doesn't receive heartbeats for 90 seconds (3 missed), it removes the service from the registry. When a service needs to call another service, it queries Eureka for the address and caches the result locally, refreshing every 30 seconds.

An important feature is self-preservation mode. If Eureka detects that more than 15% of registered services have stopped sending heartbeats, it assumes a network partition rather than mass failure. In this mode, it preserves the existing registry instead of removing services, preventing catastrophic de-registration during network issues.

Q6: What is API Gateway and what does it do?

Answer: API Gateway is a single entry point for all client requests into a microservices system. It sits between external clients and internal services, handling cross-cutting concerns so individual services don't have to.

Its responsibilities include: request routing (directing requests to the correct service), authentication and authorization (validating JWT tokens centrally), rate limiting (preventing abuse), load balancing (distributing requests across service instances), response aggregation (combining responses from multiple services into one), circuit breaking (preventing cascading failures), SSL termination (handling HTTPS externally while using HTTP internally), and logging all requests for monitoring.

Without a gateway, clients would need to know every service address, handle authentication with each service separately, and make multiple calls for composite data. Spring Cloud Gateway is the recommended tool for Spring Boot applications.

Q7: Explain Synchronous vs Asynchronous communication.

Answer: Synchronous communication means the caller sends a request and blocks, waiting for a response. It's used when you need an immediate answer, like checking product availability before placing an order. Tools include REST (using OpenFeign or WebClient) and gRPC.

Asynchronous communication means the caller sends a message to a broker and continues without waiting. The consumer processes the message when ready. It's used for operations that don't need immediate responses, like updating inventory after an order or sending notifications. Tools include RabbitMQ and Apache Kafka.

RabbitMQ is a traditional message broker that deletes messages after consumption, while Kafka is an event streaming platform that retains messages for a configured duration and supports replay. RabbitMQ is better for task queues and moderate loads; Kafka for high-throughput event streaming.

In practice, we use both: synchronous for real-time queries and asynchronous for event-driven operations.

Q8: Explain Circuit Breaker Pattern in detail.

Answer: Circuit Breaker prevents cascading failures in distributed systems. When one service is down or slow, services calling it can exhaust their resources (threads, connections) waiting for responses, eventually becoming unavailable themselves. This cascade can bring down the entire system.

The Circuit Breaker works like an electrical circuit breaker with three states: CLOSED (normal — requests pass through, failures are counted), OPEN (tripped — all requests are immediately rejected with a fallback response, no calls to the failing service), and HALF-OPEN (testing — limited test requests allowed to check if the service has recovered).

When in CLOSED state, if the failure rate exceeds a threshold (say 50% of last 10 calls), the circuit opens. In OPEN state, after a wait duration (say 30 seconds), it transitions to HALF-OPEN. In HALF-OPEN, if test requests succeed, it closes; if they fail, it opens again.

This is used alongside Retry (retry transient failures with exponential backoff), Timeout (set maximum wait time), Bulkhead (isolate thread pools per downstream service), and Fallback (return cached or default data). Resilience4j is the recommended library for Spring Boot.

Q9: Explain Saga Pattern in detail.

Answer: Saga Pattern manages distributed transactions across multiple microservices. In a monolith, we use ACID transactions with a single database. In microservices, each service has its own database, so a single transaction cannot span multiple databases.

Saga breaks a distributed transaction into a sequence of local transactions. Each local transaction updates its own database and publishes an event or message. If any step fails, compensating transactions are executed to undo previous steps.

There are two types. Choreography-based Saga: services communicate through events with no central coordinator. Each service listens for events, performs its action, and publishes new events. It's simpler for 2-3 services but becomes hard to trace and debug for complex flows.

Orchestration-based Saga: a central orchestrator directs the entire flow. It tells each service what to do, tracks progress, and handles compensations. It's easier to understand, debug, and modify, making it better for complex workflows with 4+ services.

The tradeoff of Saga is eventual consistency — the system is temporarily inconsistent between steps but eventually reaches a consistent state.

Q10: Explain CQRS and Event Sourcing.

Answer: CQRS (Command Query Responsibility Segregation) separates read and write operations into different models, often with different databases. The write side uses a database optimized for consistency (MySQL), and the read side uses a database optimized for queries (Elasticsearch). Data syncs from write to read side via events or Change Data Capture. This is useful when read operations significantly outnumber writes and when read and write patterns require fundamentally different data models.

Event Sourcing stores state changes as a sequence of events rather than storing current state. Instead of "stock = 50", it stores all the events that led to 50 (added 100, sold 20, sold 15, returned 5, sold 20). Current state is derived by replaying events. Benefits include complete audit trail, time travel (reconstruct state at any point), and event replay for debugging or recovery. Snapshots are used to optimize replay performance.

CQRS and Event Sourcing are often combined: the write side stores events (Event Sourcing), and the read side builds materialized views from those events (CQRS). However, both patterns add significant complexity and should only be used when the benefits justify the cost.

Q11: How do you handle logging and tracing in Microservices?

Answer: Centralized Logging collects logs from all services into a single place using the ELK Stack (Elasticsearch for storage and search, Logstash for collection and processing, Kibana for visualization). All services write logs to stdout in structured JSON format. Logstash collects these, processes them, and stores in Elasticsearch. Kibana provides a dashboard to search and analyze logs from all services.

Distributed Tracing tracks a request across multiple services using a unique Trace ID. When a request enters the system, a Trace ID is generated and propagated through all service calls via HTTP headers. Each service creates spans with timing information. Zipkin collects all spans and provides a visual timeline showing which services were called, how long each took, and where bottlenecks or failures occurred.

The key best practice is including the Trace ID in every log entry. This allows correlating logs from multiple services for a single request: search for a Trace ID in Kibana and see the complete request flow across all services.

Q12: What is 12-Factor App and which factors are most important for Microservices?

Answer: The 12-Factor App is a methodology with 12 best practices for building cloud-native applications, created by Heroku developers.

The most important factors for microservices are: Factor 3 (Config) — externalize configuration using Config Server or environment variables, never store credentials in code; Factor 6 (Processes) — keep services stateless, store all state in external backing services like databases or Redis, so instances can be freely added, removed, or restarted; Factor 8 (Concurrency) — scale horizontally by adding more instances, not by making one instance bigger; Factor 9 (Disposability) — services should start quickly and shut down gracefully, crucial for auto-scaling and rolling updates; and Factor 10 (Dev/Prod Parity) — minimize differences between environments using tools like Docker.

Q13: How do you secure Microservices?

Answer: Microservices security operates at multiple layers. At the external access layer, the API Gateway validates JWT tokens issued by an OAuth2 server (like Keycloak). The gateway handles authentication centrally so individual services don't have to. For authorization, JWT tokens carry user roles that services use for access control.

For service-to-service communication, we use Mutual TLS (mTLS) where both caller and target verify each other's identity, or a service mesh like Istio that handles this automatically. Secrets (passwords, API keys) are managed through HashiCorp Vault or Kubernetes Secrets, never stored in code or config files.

Additional measures include rate limiting at the gateway, HTTPS for all external communication, input validation at API boundaries, audit logging of security events, and dependency scanning for known vulnerabilities.

Q14: Explain Docker and Kubernetes in context of Microservices.

Answer: Docker packages each microservice into a container that includes the application, runtime, and all dependencies. This ensures consistency across environments — the same container runs on a developer's laptop, CI server, and production. Docker Compose is used to run multiple containers together during development.

Kubernetes orchestrates these containers in production. It handles auto-scaling (adding instances when load increases), self-healing (restarting crashed containers), rolling updates (zero-downtime deployments), service discovery (built-in DNS for service-to-service communication), and secret management. Kubernetes replaces the need for Eureka (built-in service discovery) and provides features like ConfigMaps (similar to Config Server) and Horizontal Pod Autoscaler.

In the progression: Docker creates containers, Docker Compose manages multiple containers on one machine for development, and Kubernetes manages containers across a cluster of machines in production.

Q15: When would you NOT use Microservices?

Answer: Microservices should not be used when: the application is simple (a CRUD app doesn't need distributed system complexity); the team is small (less than 8 people managing multiple services is overhead without benefit); the domain is not well understood (wrong service boundaries are costly to fix; it's better to start with a monolith and extract later when boundaries become clear); speed to market is critical (a monolith is faster to build initially); the organization lacks DevOps maturity (microservices require CI/CD, monitoring, containerization, and logging infrastructure); and when strong consistency is more important than availability (distributed transactions add significant complexity).

The key is to make the decision based on the specific needs of the project, not because microservices are trendy. Many successful companies (Shopify, Stack Overflow, Basecamp) run monoliths effectively.

Quick Revision — Key Terms for Interviews

Term	One-Liner
Loose Coupling	Services have minimal dependencies on each other
High Cohesion	Related functionality grouped together in one service
Bounded Context	A boundary within which a domain model is defined (DDD)
Eventual Consistency	Data across services will eventually be consistent, not immediately
Polyglot Persistence	Different databases for different services based on needs
Idempotency	Same request executed multiple times produces the same result
Horizontal Scaling	Adding more instances (not more resources to one instance)
Fault Tolerance	System continues operating despite component failures
Observability	Three pillars: Logs, Metrics, Traces
Infrastructure as Code	Define infrastructure (Docker, K8s) in version-controlled files
CI/CD per Service	Each service has its own build and deployment pipeline
Contract Testing	Verify services agree on API contracts without running actual services
Blue-Green Deployment	Run two identical environments, switch traffic to new version
Canary Deployment	Route small % of traffic to new version before full rollout

Term	One-Liner
Service Mesh	Infrastructure layer (Istio) handling service-to-service communication
Sidecar Pattern	Helper container alongside main container in a pod
DDD	Domain-Driven Design — approach for defining service boundaries
CAP Theorem	Distributed system can guarantee only 2 of 3: Consistency, Availability, Partition tolerance
BASE	Basically Available, Soft state, Eventually consistent (alternative to ACID)
Conway's Law	System design mirrors team communication structure

---

Comprehensive Microservices Notes — Prepared for in-depth interview preparation with Spring Boot / Spring Cloud ecosystem.

---

26. Versioning (Optimistic Locking) — Simple Diagrams

This section explains versioning with simple, easy-to-understand visuals.

Diagram 1: The Problem (Without Versioning)

Scenario: Stock has only **5 oil filters**. Two customers buying at same time.

```
        CUSTOMER A          CUSTOMER B          DATABASE
        __________          __________          ________

Step 1: Reads stock                             Stock = 5
        Sees: 5

Step 2:                     Reads stock         Stock = 5
                            Sees: 5

Step 3: Buys 2
        5 - 2 = 3
        Updates: stock=3                        Stock = 3

Step 4:                     Buys 1
                            5 - 1 = 4
                            (uses OLD value 5!)
                            Updates: stock=4    Stock = 4  ❌


   PROBLEM: Final stock = 4
   CORRECT should be: 5 - 2 - 1 = 2
   Customer A's update was LOST!
```

Result: Customer A bought 2, but DB only reduced by 1. This is called **"Lost Update Problem"**.

Diagram 2: The Solution (With @Version)

Same scenario but with version field.

```
       CUSTOMER A          CUSTOMER B          DATABASE
       __________          __________          ________
                                               Stock = 5
                                               Version = 0

Step 1: Reads stock+ver                        Stock = 5
        Sees: 5, ver=0                         Version = 0

Step 2:                    Reads stock+ver     Stock = 5
                           Sees: 5, ver=0      Version = 0

Step 3: Buys 2
        UPDATE stock=3
        WHERE version=0
        ✅ Match!                              Stock = 3
                                               Version = 1

Step 4:                    Buys 1
                           UPDATE stock=4
                           WHERE version=0
                           ❌ Version is 1!    Stock = 3
                           CONFLICT!           (unchanged)
                           Exception thrown

Step 5:                    RETRY:
                           Re-read fresh data
                           Sees: 3, ver=1
                           UPDATE stock=2
                           WHERE version=1
                           ✅ Match!           Stock = 2
                                               Version = 2


   RESULT: Final stock = 2 ✅
   Both updates applied correctly!
```

Diagram 3: Reads Are Parallel, Writes Are Sequential

Scenario: 10 users browsing same product.

```
PHASE 1: ALL READING (Parallel — No Blocking)
═══════════════════════════════════════════════

User 1  User 2  User 3  User 4  ...  User 10
  │       │       │       │             │
  ▼       ▼       ▼       ▼             ▼
┌──────────────────────────────────────────┐
│            DATABASE                      │
│       Stock = 5, Version = 0             │
│                                          │
│   All 10 reads happen TOGETHER           │
│   No waiting, super fast                 │
└──────────────────────────────────────────┘

All 10 users see: "5 in stock"


PHASE 2: ALL CLICK "BUY" (Same Time)
═══════════════════════════════════════════════

User 1  User 2  User 3  User 4  ...  User 10
  │       │       │       │             │
  ▼       ▼       ▼       ▼             ▼
┌──────────────────────────────────────────┐
│       DATABASE — ROW LOCK                │
│                                          │
│   Only 1 thread at a time can write      │
│   Others WAIT in queue                   │
└──────────────────────────────────────────┘

Order of execution:
User 1 → Updates → Stock=4, Ver=1 ✅
User 2 → Conflict → Retry → Stock=3, Ver=2 ✅
User 3 → Conflict → Retry → Stock=2, Ver=3 ✅
User 4 → Conflict → Retry → Stock=1, Ver=4 ✅
User 5 → Conflict → Retry → Stock=0, Ver=5 ✅
User 6-10 → Sees stock=0 → "Out of stock!" ❌


FINAL: 5 orders confirmed, 5 rejected
NO OVERSELL!
```

Diagram 4: How Hibernate Does It (Behind the Scenes)

You write simple code, Hibernate adds the magic.

```
YOUR CODE:
──────────
inventory.setStockQuantity(0);
inventoryRepository.save(inventory);


HIBERNATE GENERATES:
────────────────────

UPDATE inventory
SET stock_quantity = 0,
    version = version + 1
WHERE id = 1
  AND version = 0       ← MAGIC! Auto-added by Hibernate


RESULT CHECK:
─────────────

┌──────────────────────────────────┐
│  Rows affected = 1 (version matched) │
│     → SUCCESS                        │
│     → version becomes 1              │
└──────────────────────────────────┘

                OR

┌──────────────────────────────────┐
│  Rows affected = 0 (version changed) │
│     → CONFLICT!                      │
│     → OptimisticLocking Exception    │
│     → App must retry                 │
└──────────────────────────────────┘


YOU JUST ADD:
─────────────
    @Version
    private Long version;

HIBERNATE HANDLES EVERYTHING ELSE!
```

Diagram 5: 3 Locking Strategies — Quick Comparison

```
┌──────────────────────────────────────────────┐
│   1. OPTIMISTIC LOCKING (@Version)            │
│                                               │
│   "Sab thik hoga, conflict pe deal karenge"    │
│                                               │
│   ✅ Reads parallel (fast)                     │
│   ✅ Conflicts detected on write               │
│   ✅ Best for read-heavy apps                  │
│   ❌ App must handle retry logic               │
└──────────────────────────────────────────────┘


┌──────────────────────────────────────────────┐
│   2. PESSIMISTIC LOCKING (SELECT FOR UPDATE)   │
│                                               │
│   "Pakad ke rakho, koi chhede na"              │
│                                               │
│   ✅ No conflicts ever (locked exclusively)    │
│   ❌ Other reads BLOCKED (slow)                │
│   ❌ Deadlock risk                             │
│   ✅ Best for high write contention            │
└──────────────────────────────────────────────┘


┌──────────────────────────────────────────────┐
│   3. ATOMIC SQL UPDATE                        │
│                                               │
│   "DB ko ek hi statement mein karne do"        │
│                                               │
│   UPDATE inventory                            │
│   SET stock = stock - ?                       │
│   WHERE id = ? AND stock >= ?                 │
│                                               │
│   ✅ Fastest (no locks held by app)            │
│   ✅ DB does check+update atomically           │
│   ✅ Best for stock/counter operations         │
│   ❌ Limited to simple operations              │
└──────────────────────────────────────────────┘
```

Quick Summary

```
┌──────────────────────────────────────────────┐
│                                              │
│   VERSIONING in ONE LINE:                    │
│                                              │
│   "Read karte time version note karo,        │
│    write karte time check karo —             │
│    agar badal gaya toh retry karo!"          │
│                                              │
└──────────────────────────────────────────────┘


THE CORE LOGIC:

READ  →  Always parallel (no blocking)
         Returns: data + version

WRITE →  UPDATE ... WHERE id=? AND version=?
         ✓ Match → Update + version++
         ✗ No match → Exception → Retry


WHY IT MATTERS:

✅ Prevents lost updates
✅ Maintains data integrity
✅ Supports high concurrency
✅ No locks held by application
✅ Industry standard for shared state


WHEN TO USE:

• Inventory stock management
• Bank account balance
• Booking systems (seats, slots)
• Any shared mutable counter/state
```

End of Versioning Diagrams.

27. Scaling to 100,000 Concurrent Users

For small scale (10-100 users), `@Version` works fine. But **what if 100,000 users hit "Buy" simultaneously** during a flash sale? Optimistic locking alone won't survive — too many conflicts, too many retries.

This section shows how **real big tech** (Flipkart, Amazon, BookMyShow) handle massive concurrency.

Diagram 1: Why @Version Alone FAILS at 100K Scale

```
100,000 USERS click "Buy" at SAME TIME
═══════════════════════════════════════

      100,000 Threads
           │
           ▼
┌──────────────────────────────┐
│      DATABASE                │
│      Stock = 100             │
│                              │
│   ROW LOCK (one at a time!)  │
└──────────────────────────────┘
           │
           │
           │ Each thread tries to update...
           │
           ▼

What actually happens:

Thread 1:    UPDATE → Success ✅ (Stock=99)
Thread 2:    UPDATE → Conflict → Retry
Thread 3:    UPDATE → Conflict → Retry
...
Thread 100:  UPDATE → Success ✅ (Stock=0)
Thread 101:  UPDATE → "Out of stock" ❌
...
Thread 100K: UPDATE → "Out of stock" ❌


PROBLEMS:
─────────
❌ 99,900 wasted retries
❌ Database becomes hotspot
❌ Each thread waits 5-30 seconds
❌ Server CPU at 100%
❌ Memory exhausted
❌ Site CRASHES
```

Solution needed: Don't let 100K requests hit DB directly!

Diagram 2: Solution Layer 1 — Add Redis Cache

```
┌──────────────────────────────────────────┐
│  WITHOUT CACHE                           │
│                                          │
│  100K Users → Database (CRASH!)          │
│               ↑                          │
│               All hitting same row       │
└──────────────────────────────────────────┘


┌──────────────────────────────────────────┐
│  WITH REDIS CACHE                        │
│                                          │
│  100K Users                              │
│      │                                   │
│      ▼                                   │
│  ┌──────────┐                            │
│  │  REDIS   │  ← In-memory, super fast   │
│  │ Stock=100 │  ← Atomic operations      │
│  └──────────┘                            │
│        │                                 │
│        │ Reserves stock instantly        │
│        │ (microseconds)                  │
│        │                                 │
│        ▼                                 │
│  ┌──────────┐                            │
│  │ DATABASE │  ← Sync periodically       │
│  │ Stock=100 │  ← Or after order confirmed │
│  └──────────┘                            │
└──────────────────────────────────────────┘


HOW IT WORKS:
─────────────
1. Stock pre-loaded in Redis
2. User clicks "Buy"
3. Redis: DECR stock (atomic, fast)
4. If stock >= 0 → reserved
5. If stock < 0 → "Out of stock"
6. DB synced async after reservation


SPEED:
──────
Database = ~10ms per request
Redis    = ~0.1ms per request

100x FASTER!
```

Diagram 3: Solution Layer 2 — Add Kafka Queue

```
PROBLEM: Even Redis can't handle 100K requests/second
         from a single endpoint without breaking

SOLUTION: Buffer requests in Kafka, process at controlled rate


┌──────────────────────────────────────────────┐
│   100,000 Users click "Buy"                  │
│              │                                │
│              │ All requests go to Kafka       │
│              ▼                                │
│   ┌──────────────────────┐                   │
│   │      KAFKA           │                   │
│   │   ┌──────────────┐   │                   │
│   │   │ Order Queue │   │                   │
│   │   │  ┌────────┐ │   │                   │
│   │   │  │ 100K   │ │   │                   │
│   │   │  │messages│ │   │                   │
│   │   │  └────────┘ │   │                   │
│   │   └──────────────┘   │                   │
│   └──────────────────────┘                   │
│              │                                │
│              │                                │
│              │ Consumed at controlled rate    │
│              │ (e.g., 1000/sec)               │
│              ▼                                │
│   ┌──────────────────────┐                   │
│   │  Order Service       │                   │
│   │  (multiple workers)  │                   │
│   │                      │                   │
│   │  Worker 1, 2, 3...   │                   │
│   └──────────────────────┘                   │
│              │                                │
│              │                                │
│              ▼                                │
│      Redis + Database                         │
└──────────────────────────────────────────────┘


USER EXPERIENCE:
────────────────
Click "Buy" → "Order received! You'll get
               confirmation in 30 seconds" → SMS/Email later

ASYNCHRONOUS = NO BLOCKING = NO CRASHES!
```

Diagram 4: Solution Layer 3 — Stock Reservation Pattern

```
THE SMARTEST APPROACH: Reserve stock in Redis FIRST


STEP 1: User clicks "Buy"
──────────────────────────


┌──────────┐
│  USER    │
└──────────┘
     │ POST /reserve
     ▼
┌──────────────┐          ┌──────────────┐
│ Order Service│──────────►│    REDIS     │
│              │          │              │
│              │          │ Stock = 100  │
└──────────────┘          │              │
                          │ DECR stock   │
                          │ Stock = 99   │
                          │              │
                          │ Set timeout: │
                          │ "Reservation │
                          │  expires in  │
                          │  5 minutes"  │
                          └──────────────┘


STEP 2: User completes payment (within 5 min)
──────────────────────────────────────────────


┌──────────┐
│  USER    │ Payment Done
└──────────┘
     │ POST /confirm
     ▼
┌──────────────┐          ┌──────────────┐
│ Order Service│──────────►│   DATABASE   │
│              │          │              │
│              │          │ Stock = 99   │
└──────────────┘          │ (synced!)    │
                          └──────────────┘


STEP 3: User abandons cart (no payment)
────────────────────────────────────────


After 5 min, Redis auto-releases:
Stock = 99 → Stock = 100
(Other users can now buy it)


ADVANTAGES:
───────────

✅ Fair to all users (first-come-first-reserved)
✅ Prevents abandoned carts blocking stock
✅ Works with massive concurrency
✅ Used by BookMyShow, Amazon, IRCTC
```

Diagram 5: Complete Architecture for 100K Users

```
PRODUCTION-GRADE FLASH SALE ARCHITECTURE
═══════════════════════════════════════

┌──────────────────────────────────────────┐
│   100,000 Users (Mobile/Web/Apps)        │
└──────────────────────────────────────────┘
                    │
                    ▼
        ┌──────────────────────┐
        │     CDN (CloudFlare) │  ← Static assets
        └──────────────────────┘
                    │
                    ▼
        ┌──────────────────────┐
        │    LOAD BALANCER     │  ← Distributes traffic
        │   (AWS ALB / Nginx)  │     across servers
        └──────────────────────┘
                    │
     ┌──────────────┼──────────────┐
     ▼              ▼              ▼
┌─────────┐   ┌─────────┐   ┌─────────┐
│ Gateway │   │ Gateway │   │ Gateway │  ← Rate limiting
│ (10 svrs)│  │         │   │         │     per user
└────┬────┘   └────┬────┘   └────┬────┘
     │             │             │
     └─────────────┼─────────────┘
                   ▼
        ┌──────────────────────┐
        │   KAFKA (Buffer)     │  ← Absorbs spike
        │   100K msgs queued   │
        └──────────────────────┘
                    │
                    │ Consumed at 1000/sec
                    ▼
        ┌──────────────────────┐
        │   Order Service      │  ← 50 worker pods
        │   (Auto-scaled)      │
        └──────────────────────┘
                    │
     ┌──────────────┼──────────────┐
     ▼                             ▼
┌─────────┐                   ┌─────────┐
│  REDIS  │                   │ DATABASE │
│ Cache + │ ◄────sync────►    │  MySQL  │
│ Reserve │                   │ (Master) │
└─────────┘                   └─────────┘
                                   │
                                   ▼
                              ┌─────────┐
                              │ Replicas│ ← Read traffic
                              │ (10x)   │
                              └─────────┘


FLOW:
─────
1. CDN serves static content (images, CSS)
2. Load Balancer distributes 100K connections
3. Gateway rate-limits per user (max 5 req/sec)
4. Kafka buffers all "buy" requests
5. Order workers consume from Kafka
6. Each worker reserves stock in Redis (fast!)
7. After payment, sync to DB
8. DB replicas handle other read queries
```

Diagram 6: Layered Defense Strategy

```
THINK OF IT LIKE A FORTRESS
═══════════════════════════

Without defenses:
100K users → Database → 💥 BOOM

With defenses (each layer absorbs load):

┌──────────────────────────────────────────┐
│   LAYER 1: CDN                           │
│   Filters: Static content (90% of traffic)  │
│   Result: 10K reach next layer           │
└──────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────┐
│   LAYER 2: RATE LIMITER                  │
│   Filters: Spam, bots, abuse             │
│   Result: 8K legitimate users continue   │
└──────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────┐
│   LAYER 3: KAFKA QUEUE                   │
│   Buffers: Smooths out traffic spikes    │
│   Result: Steady 1K/sec to backend       │
└──────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────┐
│   LAYER 4: REDIS CACHE                   │
│   Handles: 95% of stock checks           │
│   Result: 50/sec actual DB writes        │
└──────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────┐
│   LAYER 5: DATABASE                      │
│   Final source of truth                  │
│   Handles: Manageable 50/sec load        │
└──────────────────────────────────────────┘


RESULT:
───────
100K users → Each layer filters
           → DB only sees 50/sec
           → System stays HEALTHY ✅
```

Quick Summary — Scale Solutions

```
┌──────────────────────────────────────────┐
│                                          │
│   SCALE LEVEL          SOLUTION          │
│   ────────────         ────────          │
│                                          │
│   10 users         →   @Version          │
│   100 users        →   @Version + Retry  │
│   1,000 users      →   + Atomic SQL Update │
│   10,000 users     →   + Redis Cache     │
│   100,000 users    →   + Kafka Queue     │
│   1,000,000 users  →   + DB Sharding     │
│                                          │
└──────────────────────────────────────────┘


THE KEY INSIGHT:
────────────────

At scale, NEVER hit DB directly.
Use layers to ABSORB and FILTER traffic:

1. Cache reads (Redis)
2. Queue writes (Kafka)
3. Reserve in cache, sync to DB later
4. Make user experience ASYNC


REAL-WORLD EXAMPLES:
────────────────────

• IRCTC Tatkal: Queue + Redis reservations
• BookMyShow: Seat lock in Redis (10 min)
• Flipkart Sale: Kafka + Redis + Sharded DB
• Amazon Prime Day: All layers above
• Zomato Order: Async with status callbacks


INTERVIEW GOLD:
───────────────

"For massive concurrency, we don't synchronize
 on the database. We absorb load using CDN, rate
 limiters, message queues, and in-memory caches.
 The database becomes the source of truth that
 we sync to AFTER the user-facing operation
 completes. This pattern is called eventual
 consistency with optimistic reservations."
```

End of Scaling Diagrams.